

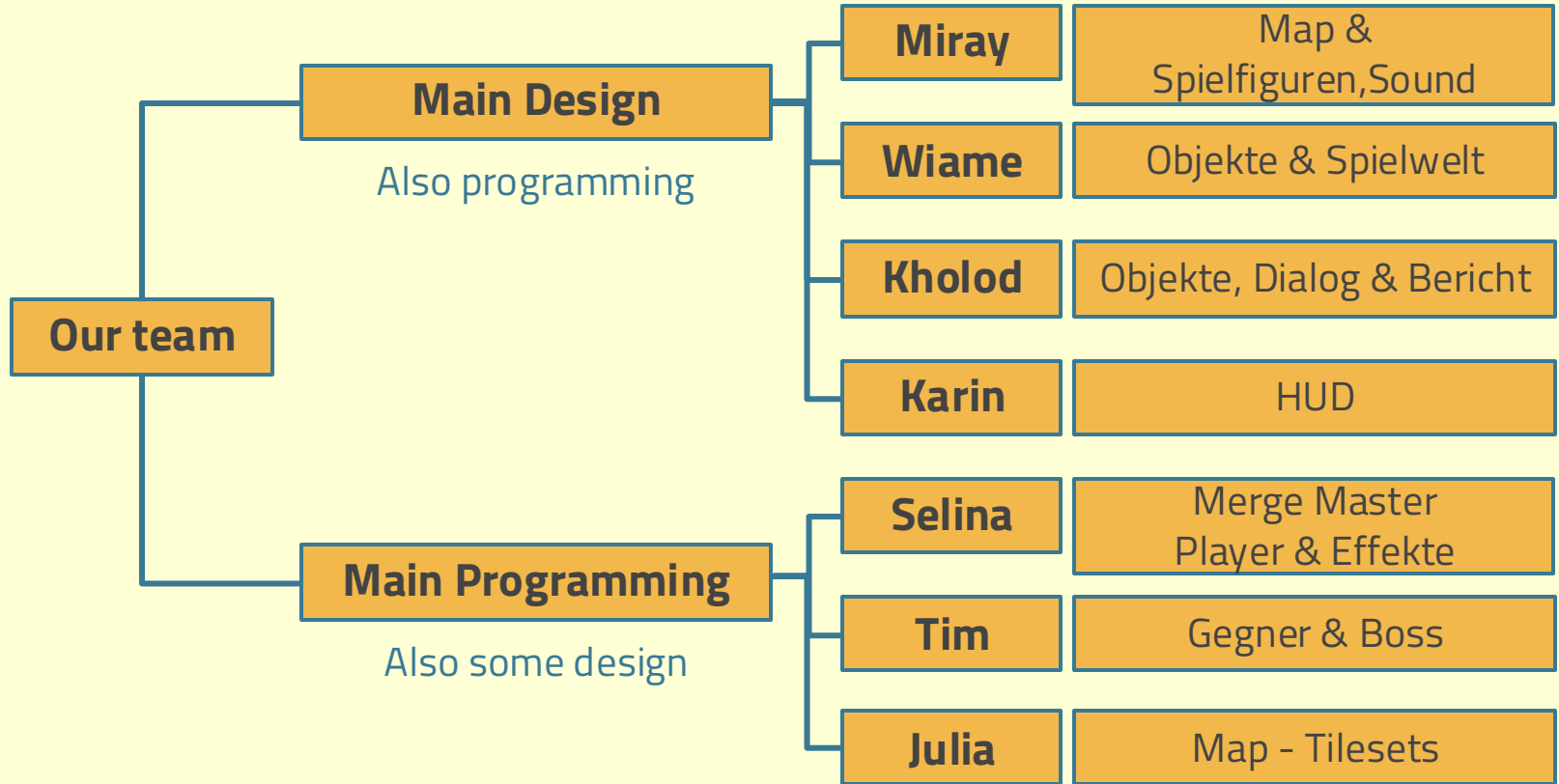
A pixel art duck is positioned on the left side of the slide. It is surrounded by several small decorative plus signs in blue, orange, and red. To the left of the duck is a vertical stack of colored squares: blue, dark blue, green, orange, and red.

Duck Duck Düsseldorf!

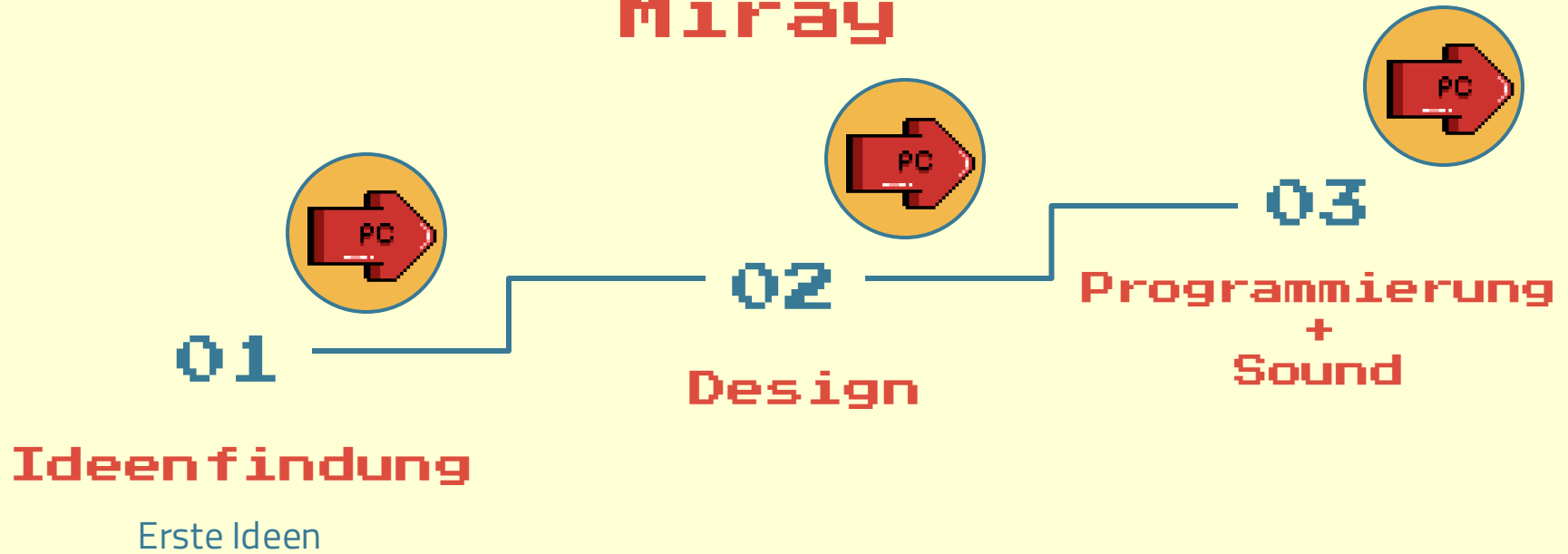
und der Entstehungsprozess

A pixel art duck is positioned on the right side of the slide. It is surrounded by several small decorative plus signs in blue, orange, and red. To the right of the duck is a vertical stack of colored squares: blue, dark blue, green, orange, and red.

Duck Duck Düsseldorf



Design & Sound Miray





01

Die Ideenfindung

Erste Ideen

1. Konzept & Game Design

- **Genre:** Survival/Apokalypse
- **Setting:** Hochschule Düsseldorf (HSD), Campus
- **Ziel:** Finde Hinweise, löse Rätsel und entkomme aus der Hochschule

2. Gameplay & Story

- Die Welt ist zusammengebrochen – man spielt als einer der letzten Überlebenden
- Ziel ist es, Hinweise zu finden (z. B. in einem Buch), um die Wahrheit hinter der Apokalypse herauszufinden
- Story-Elemente: Tagebücher, Notizen, Räume mit versteckten Hinweisen

3. Charaktere

- **Protagonist:** Student
- Weitere Charaktere:
 - Andere Studenten (mit Hinweisen oder Aufgaben)
 - Zombies (müssen in bestimmten Räumen besiegt werden – „droppen“ z. B. Gehirne)

4. Level-Design & Umgebung

- Verschiedene Räume der HSD:
 - Biolabor, Technikräume, etc.
- Map kann an reale Gebäudeteile angelehnt sein

5. Ideenansätze

Idee 1:

- Internetausfall – Studenten müssen „zombifiziert“ Wissen bei Professoren holen
- Ziel: Internet wiederherstellen

Idee 2:

- Virus wurde durch „Rubber Ducks“ ausgelöst – Spieler hilft, das Virus zu stoppen
- Zombies = Rubber Ducks
- Items wie „Enten“ dropen, die man einsammeln muss
- Items als Storyelemente (z. B. für Abschluss eines Levels)



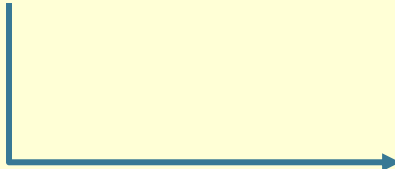
02

Das Design



Grundlegende Designs

Mapdesign



Character
-Design

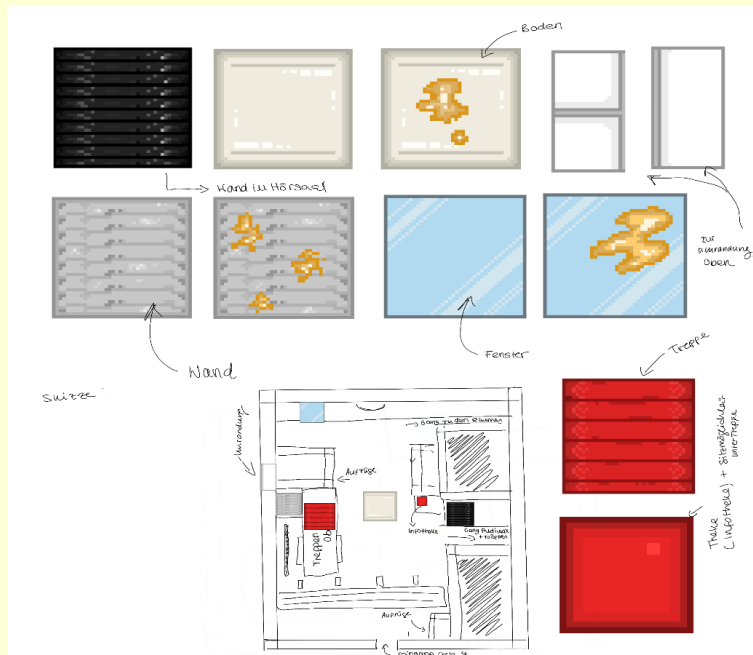


Cutscene
-Design

Mapdesign

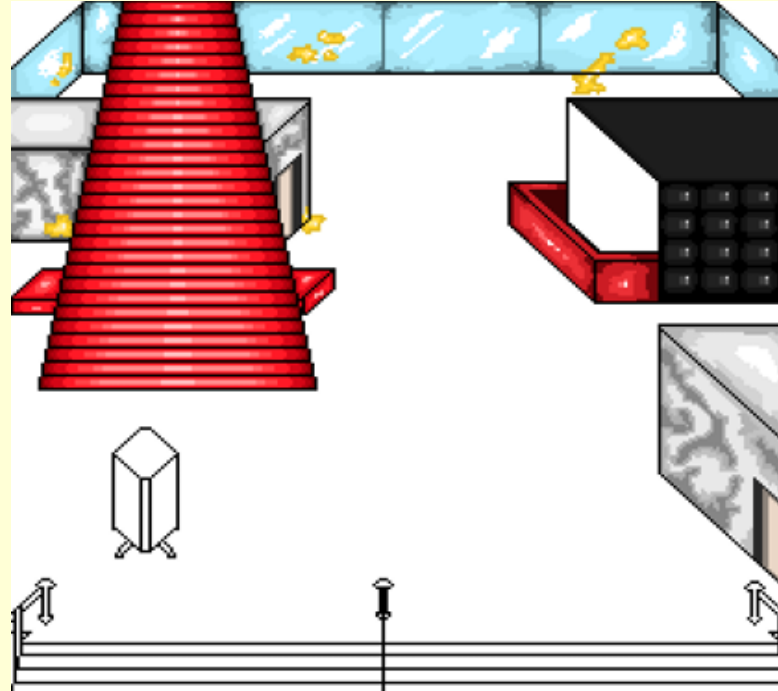
Tilemap & Tileset – Erste Schritte

- Ziel: stilistisch stimmige Grundlage für den Raum
- Erste Skizzen und Basiselemente erstellt (Boden, Wand, Übergänge)
- Fokus auf:
 - klare Strukturierung
 - Wiedererkennbarkeit
 - Modularität zur guten Kombinierbarkeit
- Erste Farb- und Stiltests zur Atmosphäre im HSD-Stil
- Nächster Schritt: detailliertere Ausarbeitung und Erweiterung des Tilesets



Mapdesign

- Szene im Format 256x256 px erstellt
- Ziel: nutzbare Tilemap für das Spiel
- Entwurf enthält tileartige Strukturen, aber kein klares 32x32-Raster
- Ergebnis: gut als Konzept, aber nicht engine-tauglich
- Nächster Schritt: Erstellung einer modularen 32x32 px-Tilemap auf technischer Basis

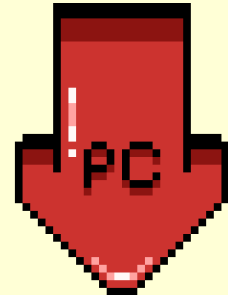
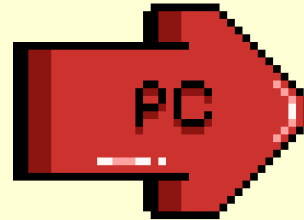


Mapdesign

- Fokus: Anpassung und Verbesserung der Tilemaps
- Neue Tiles erstellt, bestehende überarbeitet
- Ziel: visuell stimmige Spielwelt
- Verbesserte Übergänge zwischen Bereichen



Mapdesign – letzte Elemente



Die letzten Schritte zur Fertigstellung des Map-Designs wurden abgeschlossen

Character-Design



Erste Skizzen:

- Mehrere Entwürfe für den Hauptcharakter erstellt
- Ziel: visuelle Richtung und Stilfindung
- Fokus auf:
 - Gestaltung einer Ente
 - Anlehnung an Herr Dahm
 - Entwicklung einer Grundidee für die Figur

Character-Design

Charaktere – Bewegung & Animation

- Bewegungsrichtungen für alle vier Himmelsrichtungen umgesetzt
- Konsistenter Stil im Design beibehalten
- Animationen & Proportionen überarbeitet
- Sprite für Hauptcharakter fertiggestellt



Character-Design

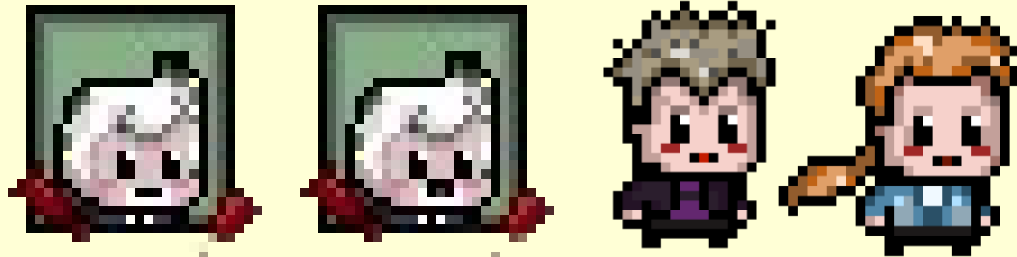
- Grundlegende Animationen für die Enten wurden erstellt & ebenso Animationen für Herr Dahm umgesetzt.

- Mehrere Designs wurden für die Mobs erstellt, um Diversität zu schaffen. Zudem wurde die Maske erstellt die den infizierten Mobs aufgesetzt wird + Endboss



Cutscene-Design

- Der Hintergrund für die Cutscene und die dazugehörige Sprechanimation des Dahm Charakters wurde fertiggestellt.
- Sprites für Müller & Antes erstellt





03

Das Programmieren und der Sound

Code – Zufälliger Gegner

- `_ready()` wird beim Szenenstart automatisch ausgeführt
 - `randomize()` sorgt für echte Zufallswerte
- Schleife versteckt alle Gegner (`visible = false, stop()`)
- Zufälliger Gegner aus `mob_sprites`-Liste ausgewählt
 - Nur dieser Gegner wird sichtbar & aktiviert

Codeausschnitt

```
func _ready():
    randomize()

    for sprite in mob_sprites:
        sprite.visible = false #Hide every mob sprite in the list - so that none are visible at the beginning
        sprite.stop() #Stops all animations

    var random_index = randi() % mob_sprites.size()
    #Generates a random number between 0 and (number of mob sprites - 1)
    #Used to select a random sprite from the list

    spriteMob = mob_sprites[random_index]
    # Selects a sprite from the mob_sprites list at random_index
    # Assigns this sprite node to spriteMob
    # spriteMob will be used throughout the entire code as the active sprite

    spriteMob.visible = true
    #only the randomly selected sprite is visible

    #We only have to change one Variable, Progress Bar adjusts automatically
    progressBar.max_value = health
    progressBar.value = health
    progressBar.hide()
```

Soundeffekte

**Soundintegration –
Schritte &
Umsetzung**

Audiodateien ins
Projekt importiert

AudioStreamPlayer
in Szenen
eingebunden

Zuweisung der
Sounds über den
Inspector

Soundeffekte per
Skript mit `.play()`
abgespielt

Anwendung z. B. bei
Buttons & Item-
Pickups

Effiziente und
einfache Sound-
Integration ins Spiel

Code-Struktur-Beispiel

zu Debuggen

Online-Dokumentation Hilfe durchsuchen

```
8  @onready var button_sound_player = $AudioStreamPlayer
9
10
11 # Notifies 'Main' node that the button has been pressed
12 signal start_game
13
14
15 # Called when the node enters the scene tree for the first time.
16
17 func _ready() -> void:
18     GlobalSignals.duck_collected_signal.connect(duck_collected_func)
19     #made names longer for claritys
20
21
22
23
24 # Called every frame. 'delta' is the elapsed time since the previous frame.
25 func _process(delta: float) -> void:
26     pass
27
28 func _on_start_button_pressed():
29     # plays pressed animation
30     start_button_animation.play("pressed")
31
32     # plays button sound when pressed
33     button_sound_player.play()
```

AudioStreamPlayer

Eigenschaften filtern

AudioStreamPlayer

Stream 665182_el_

Volume dB 0.0 dB

Pitch Scale 1.0

Playing ☐ An

Autoplay ☐ An

Stream Paused ☐ An

Mix Target Stereo

Max Polyphony 1

Bus Master

Playback Type Default

Node

> Process

> Physics Interpolation

> Auto Translate

> Editor Description

Script <leer>

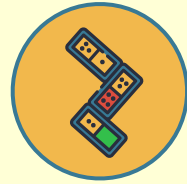
+ Metadaten hinzufügen

Objektdesign & Dialog Wiame



Raumgestal-
tung

Dialog -
System



Hintergrund-
Startscreen

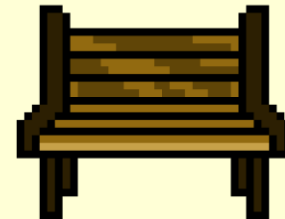
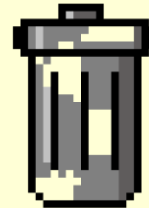


01

Design- Raumgestaltung

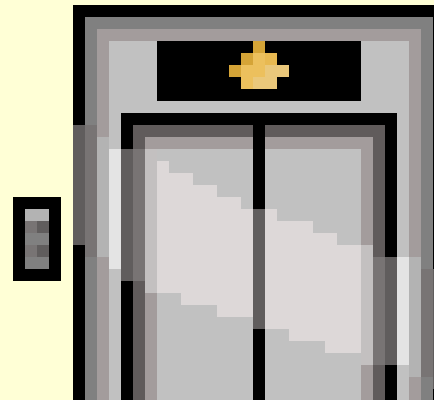
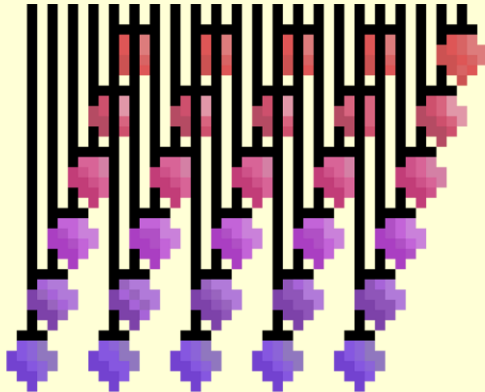
Raumgestaltung

- mehr Leben im Raum
- Raum wirkt nicht leer
- tragen zur Atmosphäre ohne direkten funktionalen Zweck
- dekorative Gegenstände
- eher lebendigere Farben



Wiedererkennungsmerkmale des Foyers

- spezifische Objekte
 - > charakteristisch für unser Foyer
 - > helfen den Spielenden, den Raum visuell wiederzuerkennen
 - > Orientierung in Realität



Gestaltung des PC-Pool

- spezifische Objekte
 - > orientiert sich an der echten Einrichtung
 - > möglichst authentische digitale Nachbildung





02

Code- Dialogsystem

Dialogsystem – Ziel & Idee

Ziel:

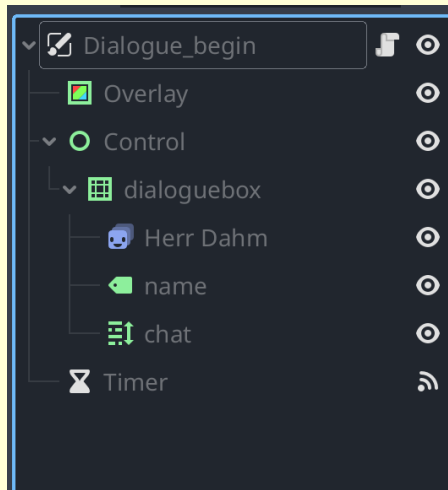
- Anfangsdialog: Einführung in die Spielwelt, Aufgabe und Ziel
- Zwischendialoge: Motivation, Fortschrittsfeedback, Tipps und Erinnerungen

Überlegungen für die Umsetzung :

- Dialogsystem selber entwickeln
 - Einblendung des Dialogfensters -> Foyer als Hintergrund
 - Möglichkeit Dialoge per Tastendruck weiterzuklicken
 - flexibles System: Inhalte über JSON leicht erweiterbar
-
- Herr Dahm als Helfer und Motivator: widerspiegelt seine Rolle als Professor
 - > begleitet durch das Spiel

Dialogfenster-/Szene

- Overlay: abgedunkelter Hintergrund -> Fokus auf Dialog
- Sprite von Herr Dahm + Dialogbox(dunkle Vorlage -> Kontrast zum Hintergrund)



Dialogsystem Umsetzung

Hauptklasse: Dialogue begin.gd:

- > Steuerung des Dialogsystems
- > lädt Texte einer .json Datei ein
- > stellt Texte mit Schreibmaschinen-Effekt, Ton und Animation dar
- wird per Signal aktiviert und am Ende automatisch beendet

```
# Path to the JSON file containing dialogue data
@export var d_file: String = ""

@onready var dialogue_box = $Control/dialoguebox
@onready var chat_label = $Control/dialoguebox/chat
@onready var name_label = $Control/dialoguebox/name
@onready var Herr_Dahm = $"Control/dialoguebox/Herr Dahm"
@onready var timer = $Timer

# Stores the parsed dialogue data from JSON
var dialogue = []
# Index to keep track of the current dialogue line
var current_dialogue_id = 0
# Flag to indicate whether the dialogue is currently active
var d_active = false
```

- lädt Dialog-Dateipfad
- Referenz zu Ui-Elementen(Name, Text, Animation)
- speichert den aktuellen Stand des Dialogs

Dialogsystem Umsetzung

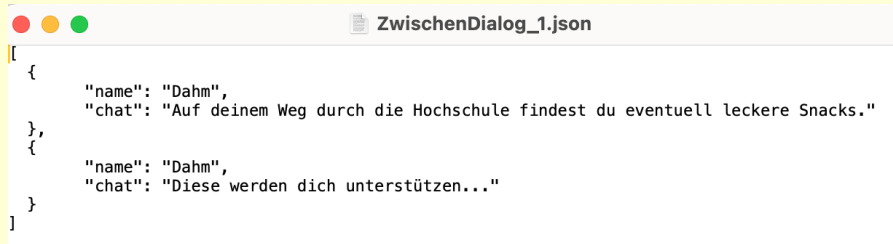
Dialog laden & parsen:

```
# Loads and parses the JSON dialogue file
▼ func load_dialogue():
▼ | if FileAccess.file_exists(d_file):
| | print("Loading file:", d_file)
| | var file = FileAccess.open(d_file, FileAccess.READ)
| | var text = file.get_as_text()
| | print("JSON content:\n", text)

| | var json = JSON.new()
| | var err = json.parse(text)
▼ | | if err != OK:
| | | push_error("Failed to parse JSON: " + text)
| | | return []

| | print("Successfully parsed JSON:", json.data)
| | return json.data
▼ | else:
| | push_error(" File does not exist: " + d_file)
| | return []
```

-liest die Dialoge aus der .json Datei
-bei Fehlern wird abgebrochen und eine leere Liste.
zurückgegeben



```
[
{
  "name": "Dahm",
  "chat": "Auf deinem Weg durch die Hochschule findest du eventuell leckere Snacks."
},
{
  "name": "Dahm",
  "chat": "Diese werden dich unterstützen..."
}
]
```

Dialogsystem Umsetzung

Dialog anzeigen:

```
# Shows the next dialogue line
func next_script():
    >| current_dialogue_id += 1

    >| # If end of dialogue is reached
    >| if current_dialogue_id >= dialogue.size():
    >| >| print("Dialogue finished.")
    >| >| d_active = false
    >| >| visible = false
    >| >| GlobalSignals.dialogue_finished.emit()
    >| >| return
    >| >|
    >| var entry = dialogue[current_dialogue_id]

    >| if typeof(entry) != TYPE_DICTIONARY:
    >| >| push_error("Invalid dialogue entry: " + str(entry))
    >| >| return
    >| >|

    >| # Get references to name and chat label
    >|
    >| # Set name and dialogue text from entry
    >| name_label.text = entry.get("name", "??")
    >| chat_label.text = entry.get("chat", "...")
    >|
```

- zeigt nächste Zeile an bis zum Ende
- Sendet Signal, wenn fertig

GlobalSignals- Verbindung zum Spiel:

```
#Dialogues that pause the game
signal dialogue_finished
signal dialogue_start
```

- _start: wird vom Spiel ausgelöst
- _finished teilt dem Spiel mit, dass der Dialog zu Ende ist

Dialogsystem Umsetzung

Zwischendialoge:

->werden automatisch getriggert über Spielfortschritte

```
#Checks if enough ducks were collected for victory
if ducks_collected == ducks_required_gameWon:
>| GlobalSignals.dialogue_start.emit("res://CutScene/Dialogue/ZwischenDialog_
>| GlobalSignals.game_won.emit()
#Starts Dialogue on half way to victory
if ducks_collected == ducks_required_gameWon / 2:
>| GlobalSignals.dialogue_start.emit("res://CutScene/Dialogue/ZwischenDialog_
```

Pausieren während Zwischendialog:

```
func _pause_level_on_dialogue_start(dialogueFile : String = ""):
>| get_tree().paused = true

func _unpause_level_on_dialogue_end():
>| get_tree().paused = false
```



02

Design- Hintergrund (Startscreen)

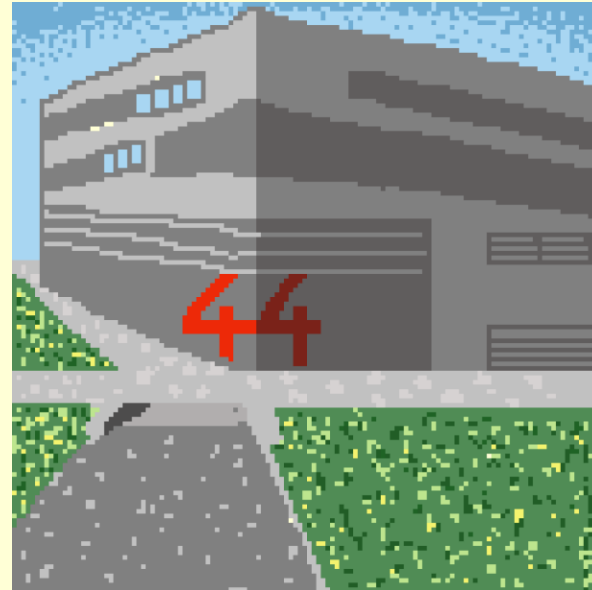
Startbildschirm

Idee: Hochschulgebäude als Pixel-Art Version

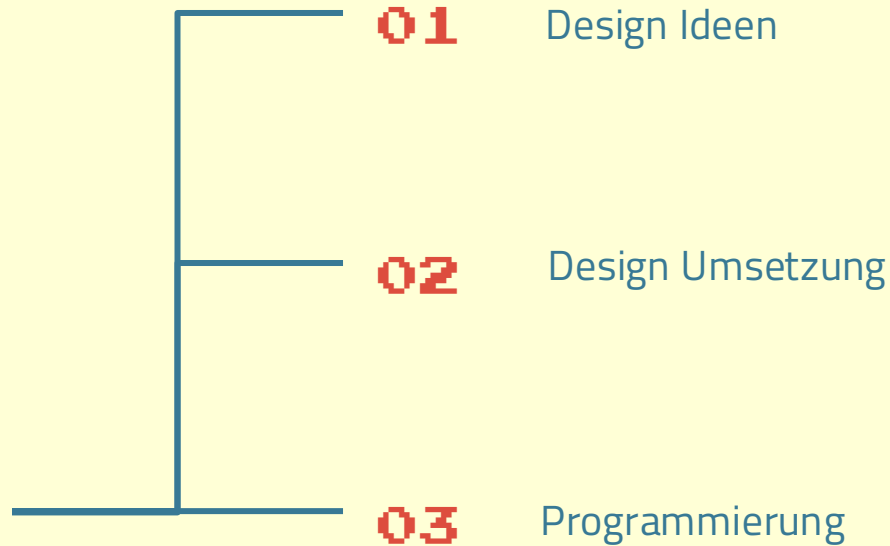
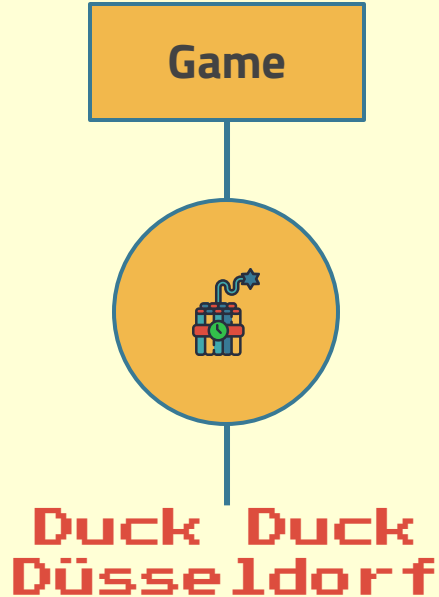
Vorlage:



Umsetzung:



Design, Dialog & Sounds: Kholod





01

Design Ideen

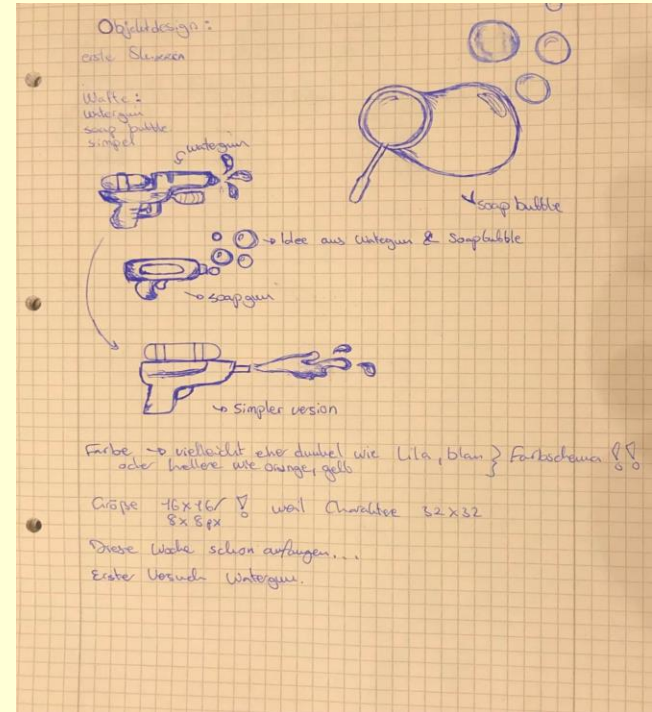
Erste Waffenideen

Wir waren uns zunächst nicht sicher, welche Waffenart wir für das Spiel verwenden wollten.

Zur Auswahl standen:

- eine Wasserpistole
- ein Seifenblasenstab

Dazu habe ich mehrere Skizzen mit verschiedenen Formen angefertigt, um unterschiedliche Designmöglichkeiten zu erkunden.





02

Design Umsetzung

Waffen Design

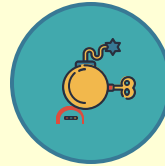
Wasser- pistole

Erster Entwurf



Seifen- blasenstab

Zweiter Entwurf



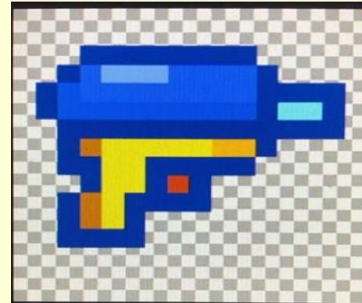
Wasser- tropfen

Als Projektile



Wasser- pistole

Finales Ergebnis



Drops-Design

Kantinen essen

Für Lebens-
generation



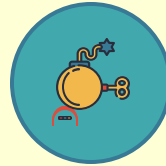
Energy drink

Für die
Schussrate



Snack- bar

Für Speedboost



Charakterdesign (Entwurf)



Student
Von vorne



Student
Von hinten



Charakterdesign:

Der erste Student-Charakter wurde im Stil der anderen Figuren gestaltet. Form, Größe und Stil sind bewusst einheitlich. Die Skizze dient als Grundlage für weitere Charaktere.

Student
Von der Seite





03

Programmierung

Drops (Basisversion)

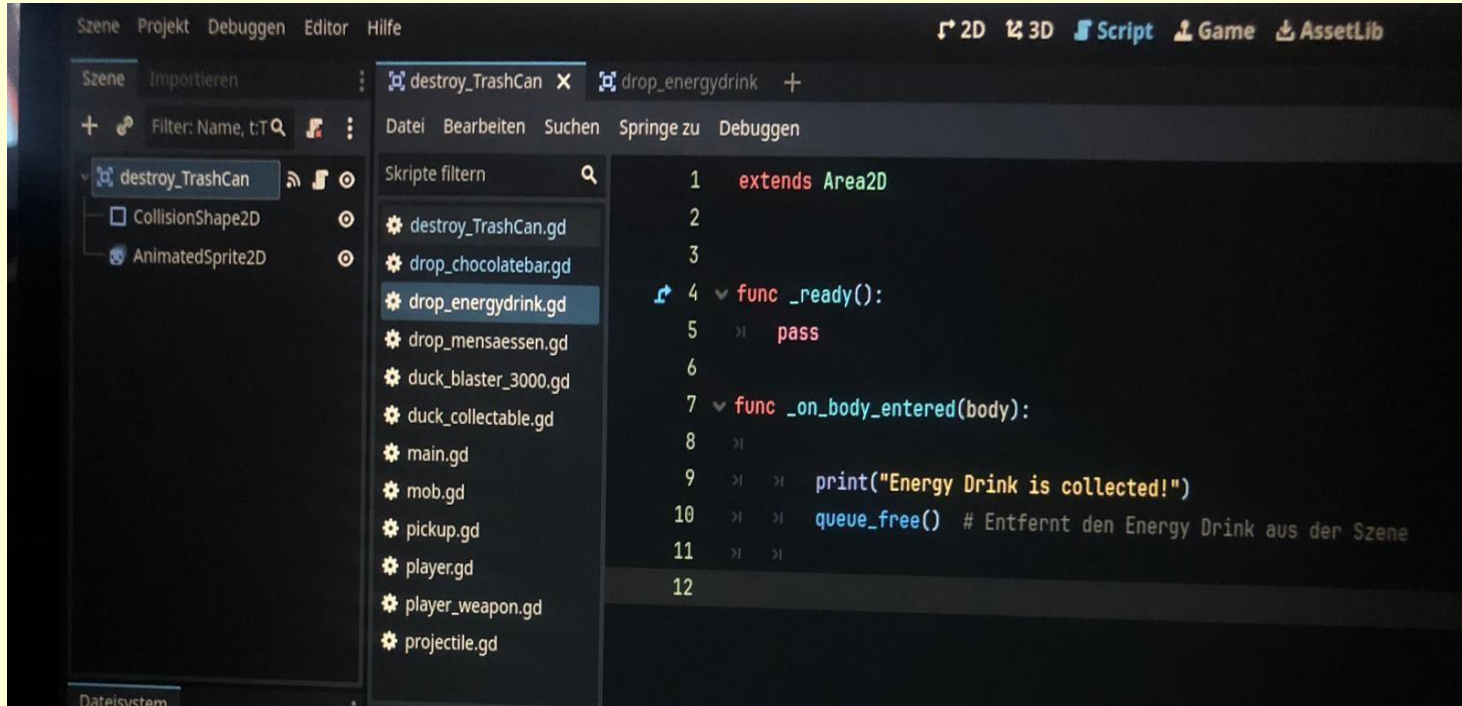
Idee:

Spieler soll beim Überlaufen eines Drops eine Rückmeldung bekommen → Pickup-Mechanik als Basisfunktion.

Umsetzung:

- Script für Energy Drink, Chocolatebar und Kantinenessen geschrieben (extends Area2D)
- Reagiert auf Kollision mit Spieler (`_on_body_entered`)
- Gibt Nachricht in Konsole aus z.B („Energy Drink is collected!“)
- Entfernt sich danach mit `queue_free()` aus der Szene

Codeausschnitt



Interaktive Mülltonne mit Item Drop (Grundlogik)

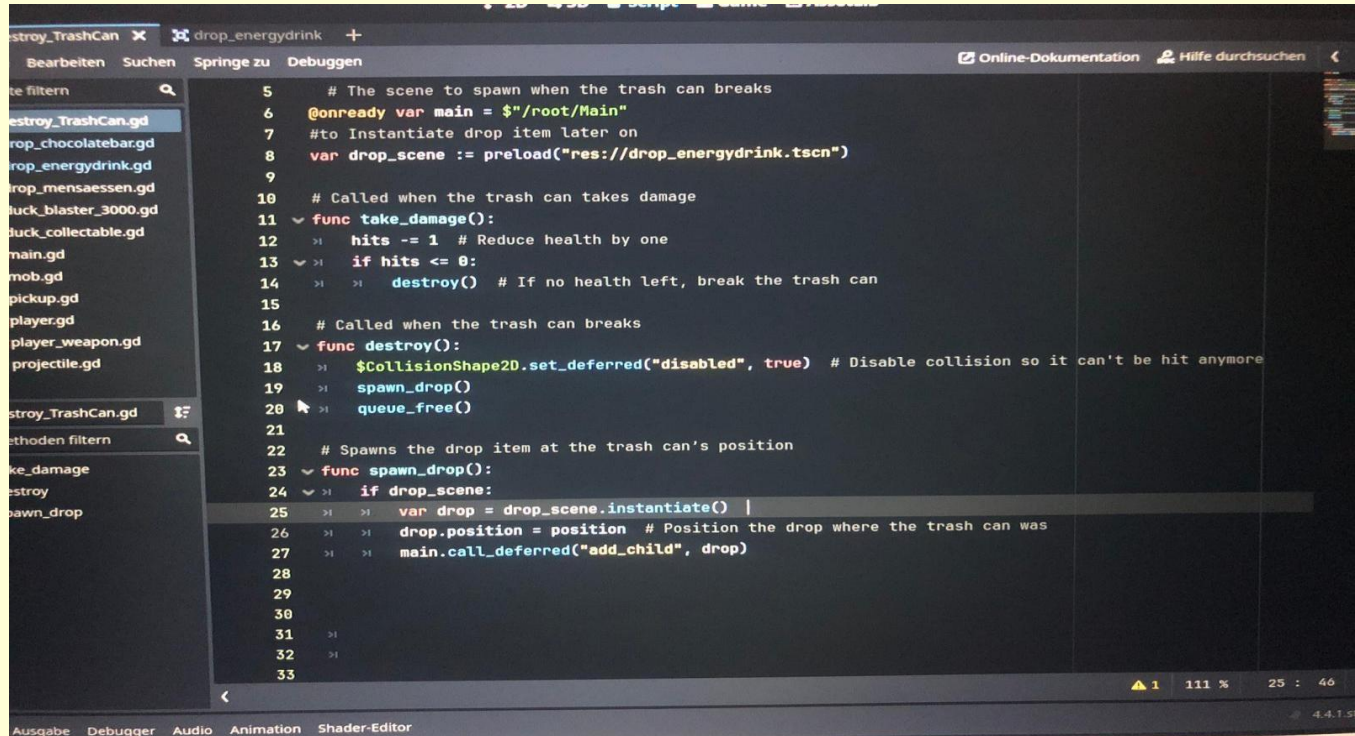
Idee:

Objekte im Spiel sollten zerstörbar sein und bei Zerstörung automatisch ein Item dropen – z. B. einen Energy Drink.

Umsetzung:

- Area2D-basierte Mülltonne mit einfachem Lebenspunktsystem (hits)
- Bei Treffern: `take_damage()` reduziert Lebenspunkte
- Wenn 0 Punkte → `destroy()` wird aufgerufen:
- Kollision deaktivieren, um weitere Treffer zu blockieren
- Drop-Szene (z. B. `drop_energydrink.tscn`) mit `preload()` vorbereitet
- Drop wird per `call_deferred("add_child")` in Hauptszene eingefügt
- Mülltonne entfernt sich mit `queue_free()`

Codeausschnitt



The screenshot shows the Godot 4.4.1 script editor with a script named `destroy_TrashCan.gd`. The code is written in GDScript and defines the behavior of a trash can when it takes damage or is destroyed. The script includes comments in German and uses Godot's built-in functions for scene management and collision handling.

```
5 # The scene to spawn when the trash can breaks
6 @onready var main = $"/root/Main"
7 #to Instantiate drop item later on
8 var drop_scene := preload("res://drop_energydrink.tscn")
9
10 # Called when the trash can takes damage
11 func take_damage():
12     hits -= 1 # Reduce health by one
13     if hits <= 0:
14         destroy() # If no health left, break the trash can
15
16 # Called when the trash can breaks
17 func destroy():
18     $CollisionShape2D.set_deferred("disabled", true) # Disable collision so it can't be hit anymore
19     spawn_drop()
20     queue_free()
21
22 # Spawns the drop item at the trash can's position
23 func spawn_drop():
24     if drop_scene:
25         var drop = drop_scene.instantiate() |
26         drop.position = position # Position the drop where the trash can was
27         main.call_deferred("add_child", drop)
28
29
30
31
32
33
```

The interface includes a file explorer on the left with a search bar, a top menu bar with options like 'Bearbeiten', 'Suchen', 'Springe zu', and 'Debuggen', and a status bar at the bottom showing 'Ausgabe', 'Debugger', 'Audio', 'Animation', 'Shader-Editor', and the version '4.4.1.5'.

Dialog Endszene

Idee:

Für die Endszene sollte ein Dialog ablaufen, bei dem ein einzelner Charakter (Herr Dahm) Text spricht. Die Anzeige sollte animiert, steuerbar und gut lesbar sein.

Umsetzung:

- Dialogdaten aus externer .json geladen (load_dialog())
- Mit next_script() wird der aktuelle Text gesetzt:
 - Name & Text werden angezeigt
- Mit Leertaste oder Enter springt man zur nächsten Zeile
- Nach dem letzten Eintrag blendet sich die Dialogbox aus

```
40  > > > "RESET":
41  > > > GlobalSignals.scene_controller.change_gui_scene("res://Hud/starts
42
43  # starts the dialogue sequence.
44  > func start():
45  > > dialogue = load_dialogue() # loads the dialogue from the JSON file
46  > > current_dialogue_id = -1 #resets the dialogue index
47  > > Herr_Dahm.play("open") #plays the starting animation
48  > > next_script() #moves to the first dialogue line
49
50  # loads the dialogue data from the specified JSON file.
51  > func load_dialogue():
52  > > # Loads and parses JSON file
53  > > if FileAccess.file_exists(d_file): #If the file exists
54  > > > #reads and parses the JSON content
55  > > > var file = FileAccess.open(d_file, FileAccess.READ)
56  > > > return JSON.parse_string(file.get_as_text())
57  > > > return [] #Otherwise, it returns an empty array
58
59  #handles player input during the dialogue sequence.
60  > func _input(event):
61  > > if not d_active:
62  > > > return
63  > > if event.is_action_pressed("skip_cutscene"): # allows to skip
64  > > > $fade_in/AnimationPlayer.play("RESET") # Resets fade-in animation
```

Ein Teil vom Code

Typing-Effekt & Sound

Idee:

Der Text sollte nicht sofort komplett erscheinen, sondern Buchstabe für Buchstabe eingeblendet werden, begleitet von Ton und passender Mundanimation, um den Monolog realistischer und filmischer wirken zu lassen.

Umsetzung:

Einen Timer eingerichtet, der alle 0.055 Sekunden den nächsten Buchstaben anzeigt (`_on_timer_timeout`)

Während der Text geschrieben wird:

- Der Schreibsound (`typing_sound`) wird abgespielt
- Die Figur **Herr Dahm** spielt eine Mundbewegung (Animation: "open")

Sobald der ganze Text sichtbar ist:

- Der Effekt stoppt automatisch
- Der Sound wird beendet
- Die Figur kehrt zur "default"-Animation zurück (Mund geschlossen)

Typing Effect & Sound

```
# Typing effect: reveals characters one by one each frame
✓ func _on_timer_timeout() -> void:
✓ >| if not typing: # if it's not typing then stop the timer
>| >| timer.stop()
>| >| return

>| char_index += 1 #else add a character every 0.05 seconds
>| textbox_text.visible_characters = char_index # update

✓ >| if char_index <= textbox_text.text.length():
>| >| Herr_Dahm.set_deferred("animation", "open") #Plays Animation aslong as text is typed
>| >| # if there are still characters left play sound
>| >| GlobalSignals.play_sound.emit("typing_sound")
>| >| #stop sound and typing when ther are no characters left
✓ >| if char_index >= textbox_text.text.length():
>| >| typing = false
>| >| timer.stop()
>| >| GlobalSignals.stop_sound.emit("typing_sound")
>| >| Herr_Dahm.set_deferred("animation", "default") #Stops Mouth movement
```

Zentrale Soundssteuerung über GlobalSignals

Idee:

Soundeffekte (z. B. Tippgeräusche im Dialog) sollten zentral gesteuert werden ohne jedes Mal eigene Soundfunktionen in jeder Szene zu schreiben. Dafür wurde ein GlobalSignals-System verwendet.

Umsetzung:

- Ein zentrales Skript verbindet globale Signale mit lokalen Funktionen:
 - play_sound: Startet Sound über einen AudioStreamPlayer
 - stop_sound: Beendet Sound gezielt
- Mit:
 - `GlobalSignals.start_sound.emit("sound name")`
 - `GlobalSignals.stop_sound.emit("sound name")`
 - Kann überall abgespielt oder gestoppt werden.

```
1  extends Node
2  # is called automatically when the scene is ready
3  func _ready():
4      >| #To connect global play and stop signals to local functions
5      >| GlobalSignals.play_sound.connect(_play_sound)
6      >| GlobalSignals.stop_sound.connect(_stop_sound)
7      >|
8      #is called when the "play_sound" signal is emitted.
9  func _play_sound(name: String):
10     >| # Makes sure it's an AudioStreamPlayer and it's not already playing
11     >| if has_node(name):
12         >| var player = get_node(name)
13         >| #Only starts sound if none is playing
14         >| if player is AudioStreamPlayer and not player.playing:
15             >| >| player.play() # plays sound
16         >| >|
17     #is called when the "stop_sound" signal is emitted.
18 func _stop_sound(name: String):
19     >| # 'name' is the name of the AudioStreamPlayer node to stop.
20     >| if has_node(name):
21         >| # Checks if the node exists
22         >| var player = get_node(name)
23         >| # Ensures it's an AudioStreamPlayer and stop it
24         >| if player is AudioStreamPlayer:
25             >| >| player.stop() #stops sound
26
```

Globalsignal Beispiel

```
139
140 ▼>| if char_index <= chat_label.text.length():
141     >| >| Herr_Dahm.set_deferred("animation", "open") #Plays Animation aslong as text is typed
142     >| >| # if there are still characters left play sound
143     >| >| GlobalSignals.play_sound.emit("typing_sound")
144     >| #stop sound and typing when ther are no characters left
145 ▼>| if char_index >= chat_label.text.length():
146     >| >| typing = false
147     >| >| timer.stop()
148     >| >| GlobalSignals.stop_sound.emit("typing_sound")
```



UI-Karin

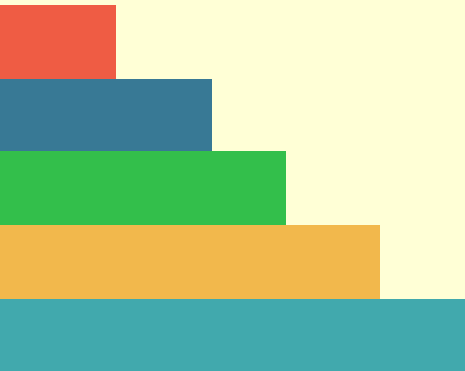


01

Das Design

Farbpalette

- Helle und freundliche Atmosphäre
- Basis für alle weiteren UI-Elemente



Start-button Design

- Erste Konzeptskizze vor der finalen Farbpalette erstellt



- Komplette Neugestaltung nach Farbschema-Entwicklung



- Nicht nur Farben geändert, sondern ganzes Design für Animationen optimiert



Button Design

- Einheitliches Design mit Start Button beibehalten
- Anpassung an das rote Farbschema des Game Over Screens

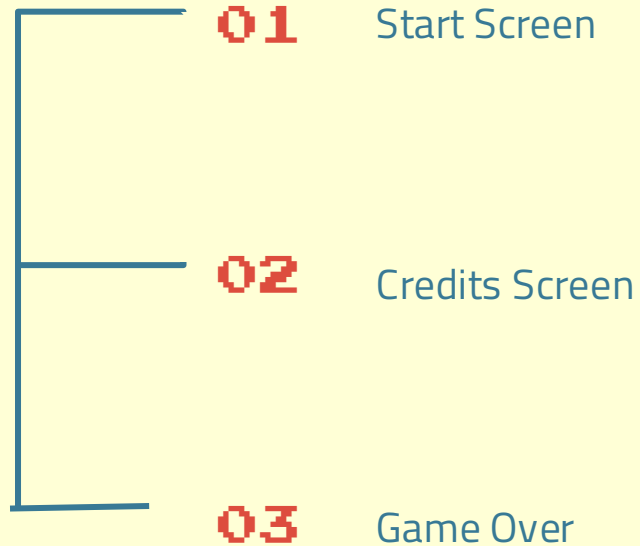


- Visuelle Differenzierung zu spiel-bezogenen Buttons



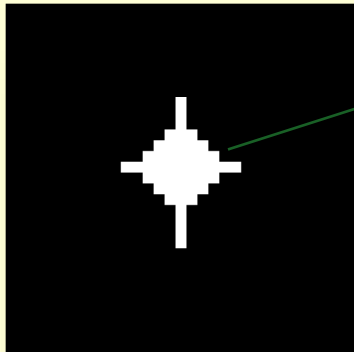
UI-Design

UI-Screens



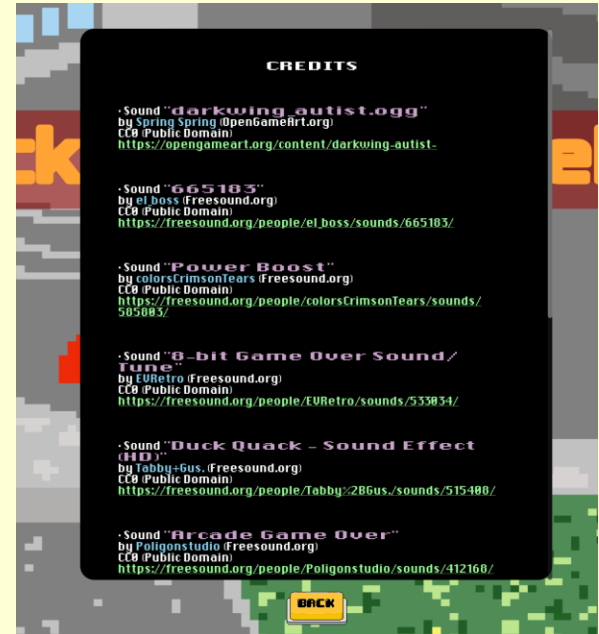
Start Screen

- Hintergrund: Campus-Gebäude (Wiame)
- Unter allen Buttons gibt es ein teilweise transparentes, farbiges Panel, um die Aufmerksamkeit zu ziehen
- Für den Titel wurde derselbe Font benutzt wie auch für den Item-Zähler im Game
- Auf dem Titel gibt es auch eine kleine Shine-Animation, um den Start-Screen etwas lebendiger zu machen



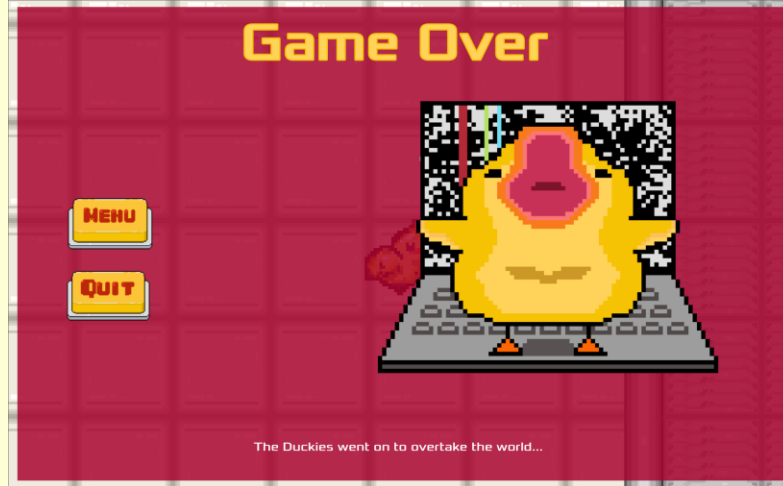
Credits-Screen Design

- Erscheint als Overlay über dem Start Screen
- Farbkodierung-system
- Medientyp, Autor, Website-Link, Lizenz



Game Over-Screen Design

- Asset einer Ente erstellt, die aus Computer herauskommt
- Verwendung der zuvor erstellten Buttons
- Gesamter Game-Over-Bildschirm erscheint über der Karte
- Bildschirm ist leicht transparent



In-Game Assets

- Gesundheitsbalken für den Spieler und die Gegner
- bestehen aus dem "Container" und der Füllung



- Font für den Zähler für die gesammelten Enten

• 0123456789

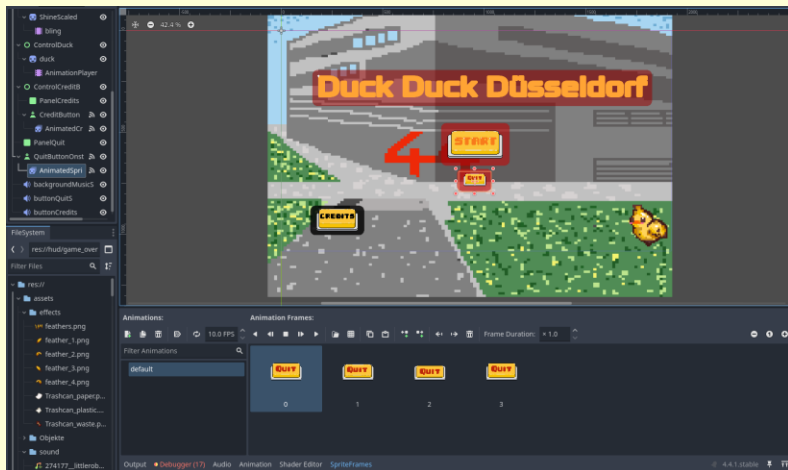


02

Umsetzung

Buttons

- Alle Buttons sind mit dem gleichen Prinzip umgesetzt
- AnimatedSprite2D + Sprite Sheet
- `_on_button_pressed()` + `_on_animation_finished()`



```
1 extends CanvasLayer
2
3 ## FUNCTIONS MENU BUTTON
4 func _on_menu_pressed() -> void:
5     GlobalSignals.play_sound.emit("starting_sound")
6     $ControlMenu/Menu/AnimatedMenuButton.play("pressed")
7
8
9
10 func _on_animated_menu_button_animation_finished() -> void:
11
12     GlobalSignals.scene_controller.remove_scene()
13     GlobalSignals.scene_controller.change_gui_scene("res://Hud/startscreen.tscn")
14
15
16 ## FUNCTIONS QUIT BUTTON
17 func _on_quit_pressed() -> void:
18
19     GlobalSignals.play_sound.emit("starting_sound")
20     $ControlQuit/Quit/AnimatedQuitButton.play("pressed")
21
22
23
24 func _on_animated_quit_button_animation_finished() -> void:
25
26     get_tree().quit()
```

Code Strukturierung Beispiel

Start-, Game Over- und Credits-Screen

- CanvasLayer
- Alle werden mit dem Scene Controller gesteuert
- Start Screen wird vom Anfang an initialisiert
- Der Game Over Screen
 - Player --- game over signal ---> _game_over() in Level Manager --- shows ---> Game Over Screen
- Der Credits Screen wird mit dem Credits Button aufgerufen

```
## FUNCTIONS
func _ready():
    #Sets global variable to this class, functions can be called globally
    GlobalSignals.scene_controller = self
>
>
> #This is what we start with; our start screen!
> GlobalSignals.scene_controller.change_gui_scene("res://Hud/startscreen.tscn")
```

Code Ausschnitt Beispiel

```
func _on_animation_player_animation_finished(anim_name: StringName) -> void:
>| if anim_name == "scale":
>| >| #global game over
>| >| GlobalSignals.game_over.emit()
>| elif anim_name == "game_won":
>| >| #Emits signal to stop enemies from spawning on this floor
>| >| GlobalSignals.toggle_natural_spawns.emit()
>| >| #Reanimates Player and his weapon
>| >| _disable_player_interactions(false)
```

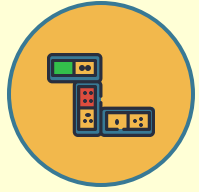
Player Skript



```
func _game_over():
>| #resets ducks collected for new game attempt
>| duck_reset()
>| #Calls on end screen as overlay over main and pauses/hides ingame overlay
>| #hiding/pausing avoids reload on button pressed "start game anew"
>| GlobalSignals.scene_controller.change_gui_scene("res://Hud/game_over.tscn", false, false)
```

Level_manager Skript





Map Umsetzung: Julia

Kurze Einleitung

Ebenen



- Navigation Layers
- Mehrschichtig
- 2 Maps in Scene

Räume



- Physics Layers
- Überdachung & Betretbar

Aufzug



- Area2D
- CollisionShape 2D
- Individuelles Script

Strukturierung



01

Node2D als Parent & umfasst zwei Map-Flächen

02

TileMapLayer nach Anwendungen aufgeteilt

03

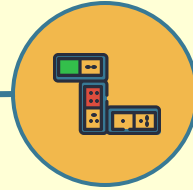
Weitere unterteilungen für unterschiedliche z-Indexe

04

Area2D für collision detection (relevant für Scripte)

TileMapLayer Properties

Navigation
Layers



Physics
Layers

Verhalten

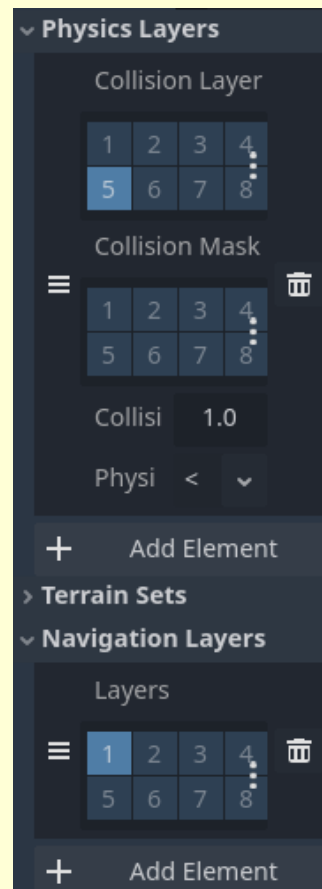
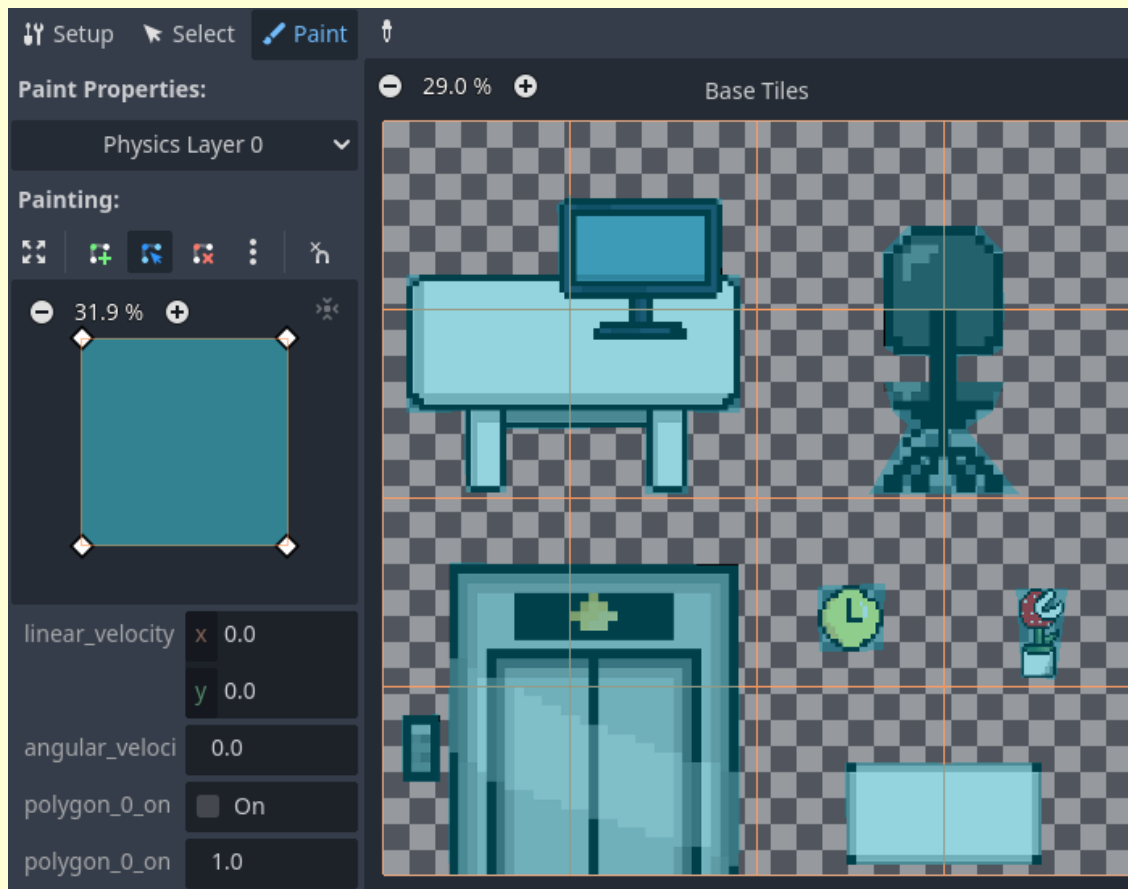
Eigenschaft

- Gegner können sich innerhalb dieses Feldes bewegen und spawnen
- Nicht begehbare Bereiche werden rausgenommen

- Ausgewählte Elemente bekommen einen Kollisionsbereich



Bearbeitung



Map spezifische Scripte



fade_in: betreten Collision2D

fade_out:
verlassen Collision2D

**fade_on
roof.gd**



Aufzug taucht erst auf, wenn die vorletzte
Ente gesammelt wurde

Durch den teleport ist der Wechsel
zwischen den Etagen möglich

**elevator_
portal.gd**



Navigation Layers Anpassung

spawn_check → Tim

boden.gd



Script-Ausschnitt

Filter Scripts

fade_on_roof.gd

fade_on_roof.gd

Filter Methods

_ready

_process

_check_fade

_set_endScene_flag

_on_body_entered

_on_body_exited

```
1  class_name fadeIN_and_fadeOut
2  extends Area2D
3
4  @onready var dach: TileMapLayer = $"../Dach_fade"
5
6  var fading_in: bool
7  var fading_out: bool
8
9  var endScene_enabled : bool = false #Gets changed with a Signal
10
11  const fade_speed: float = 5.0 # controls the rate at which the fading occurs
12
13  #Connects to boss_slain Signal to enable dialogue once game is finished
14  func _ready() -> void:
15      GlobalSignals.boss_slain.connect(_set_endScene_flag)
16
17  # Called every frame. 'delta' is the elapsed time since the previous frame.
18  func _process(delta: float) -> void:
19      if fading_in:
20          ## the code increases the alpha (transparency) value of the dach element by adding delta * fade_speed to it
21          dach.modulate.a += delta * fade_speed
22          ## once the alpha value reaches 1.0 (fully opaque), the fading_in flag is set to false
23          if dach.modulate.a >= 1.0:
24              fading_in = false
25          if fading_out:
26              ## the code decreases the alpha value of the dach element by subtracting delta * fade_speed from it
27              dach.modulate.a -= delta * fade_speed
28              ## once the alpha value reaches 0.0 (fully transparent), the fading_out flag is set to false
29              if dach.modulate.a <= 0.0:
30                  fading_out = false
```


Kleines Extra: Ent-Gegner

Schnupperaufgabe

Design und animation

01

Design

Inspiriert von den kleinen Enten

02

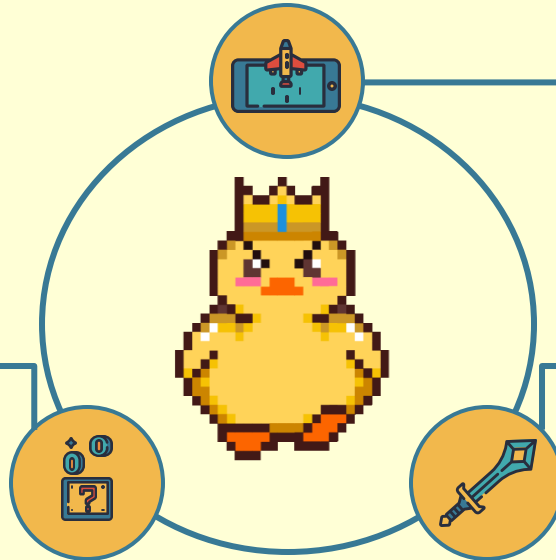
Umsetzung

Mechanik & Funktionen von Tim entwickelt

03

Final

Selina hat die Daten endgültig eingebaut, da ich irgendwann feststeckte



Fazit zur Map-Bearbeitung

Zufrieden

- Man erkennt das Gebäude 4
- Map-Eigenschaften funktionieren



Verbesserungen

- Mehr begehbbare Räume
- Umsetzung der restlichen Aufzüge



Mobs-Erklärung: Tim



Wie geht es
besser?

Boss

Umsetzung

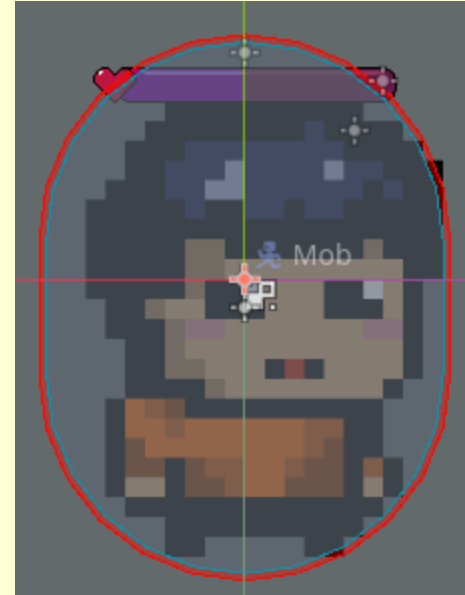
Wie haben wir es
implementiert?

Was waren unsere
Ziele/Ideen?

Grundideen

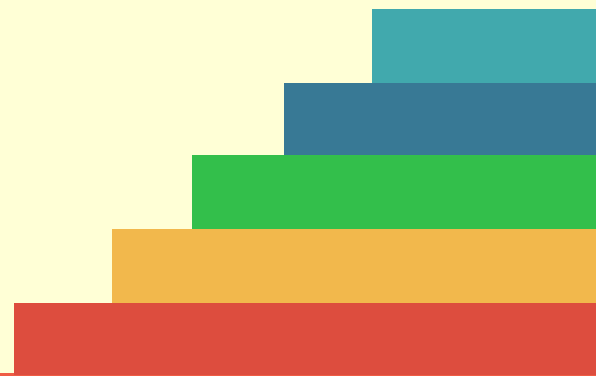
Funktionale Grundideen

- Vampire Survivors like behaviour:
 - o Pathing zum Spieler
 - o Meelee & Ranged Gegner
 - o Spawning am Bildschirmrand
- StudENTEN befreien
- Wasserpistolen für Spieler & Ranged Mobs





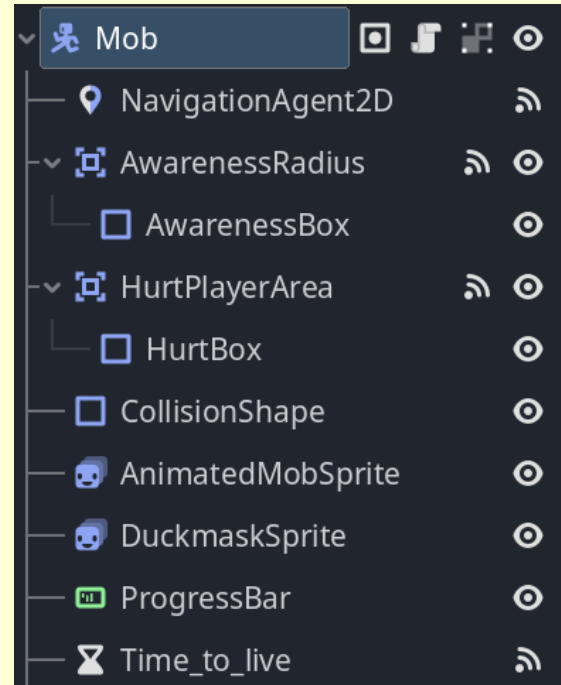
Umsetzung



Mob/Ranged Mob Scenes

Hauptbestandteile der Mob Scenes:

- 3x Area2D
 - Awareness: Sehe ich den Spieler?
 - Damage: Berühre ich den Spieler?
 - Health + Collision: Wurde ich getroffen? + Kann Ich hier laufen?
- Sprites
 - Zufällig ausgewählt
 - Maske zeigt ob Student befreit wurde
- ProgressBar
 - Wieviel Leben habe ich?
- Time_to_live
 - Wann despawne ich nachdem ich befreit wurde?
- NavigationAgent2D
 - Wo darf ich lang laufen?



Stats und Hilfsvariablen

```
## STATS
static var moblvl = 1
var health : int = 3 + (moblvl-1) * 2 #Hits r
var movement_speed = 75 + (moblvl-1) * 20
var damage_rate : float = 2.5 + moblvl * 2.5

## TARGETS
var target_damage : Node2D #saveslot for Play
var target_homing : Node2D #saveslot for curr
var duck_status : int = 1 #modifier for runni

## BOSS FIGHT CHANGES
const BOSS_MOVEMENT_SPEED = 95
static var dont_drop_duck: bool = false
```



Mob Movement

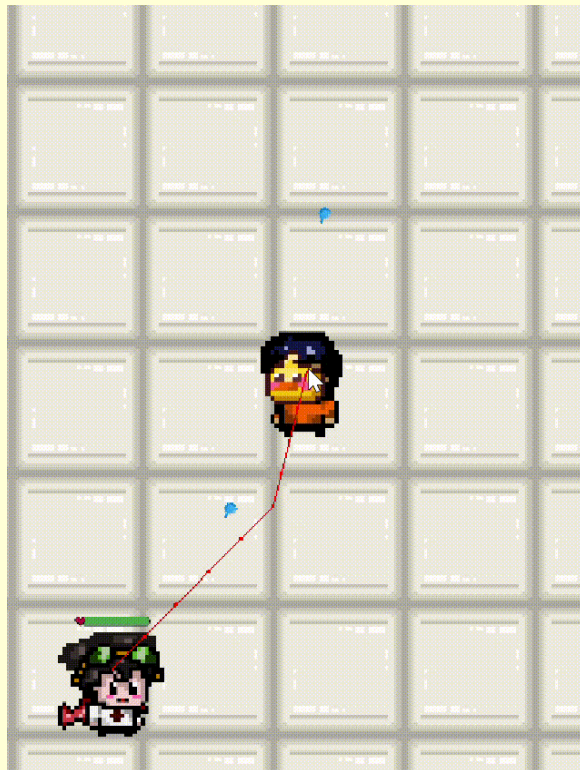
```
## MOVEMENT
velocity = Vector2.ZERO
#target_homing is the Player Node (checked in Signals) and calls on Player Position
if target_homing:
>|  var target_location = target_homing.global_position
>|  navigation_agent_2d.target_position = target_location
>|
>|  var current_agent_position = self.global_position
>|  var next_path_position = navigation_agent_2d.get_next_path_position()
>|  var new_velocity = current_agent_position.direction_to(next_path_position) * movement_speed * duck_status
>|
>|  #if navigation_agent_2d.is_navigation_finished():
>|  #>|  return #could add some kinda mellee hit animation
>|
>|  if navigation_agent_2d.avoidance_enabled:
>|  >|  navigation_agent_2d.set_velocity(new_velocity)
>|  else:
>|  >|  _on_navigation_agent_2d_velocity_computed(new_velocity)
>|
>|  move_and_slide()
```


Studenten befreien

Studenten werden befreit über die `liberated()` Funktion.

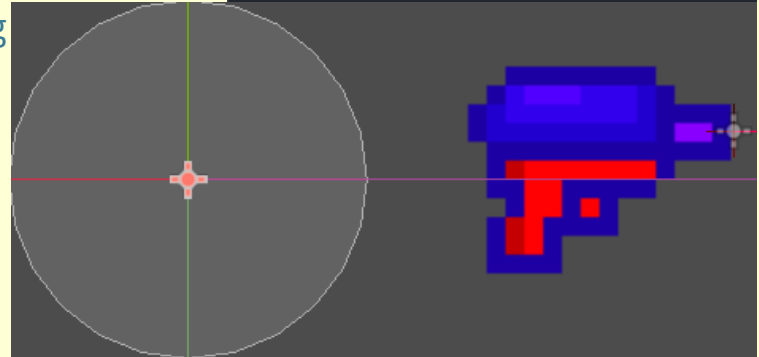
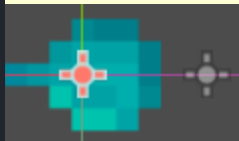
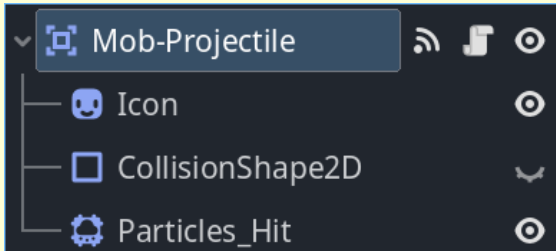
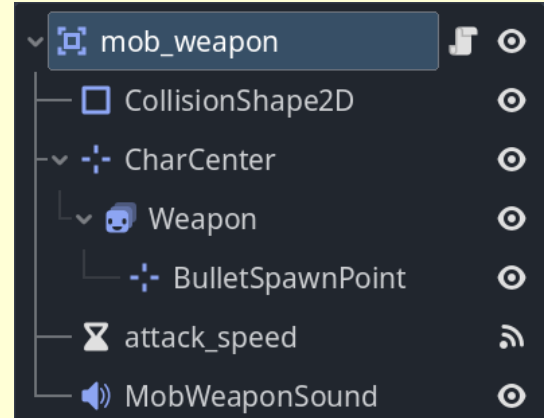
Was tut diese Funktion?

- Setzt `duck_status` zu `-1`
 - o `duck_status` ist Standardmäßig 1 (auf Spieler zulaufen)
- Versteckt das Sprite der Entenmaske
- Deaktiviert die `HurtPlayerArea` & `Mob_Weapon(ranged mob)`
- Ruft eine Methode für einen Item Drop auf



Mob/Player Weapon Scenes

- Rotiert um den Ranged Mob
- Zielt in die Richtung, in die sich der Mob bewegt
 - o also wenn möglich auf den Spieler
- Attack Speed Timer bestimmt Abstand zwischen jedem Schuss
- Mob Projectiles spawnen am Ende der Waffe bei jedem timeout
- Projectiles:
 - o sind Top-Level sodass sie sich nicht mit dem Mob bewegen
 - o Rotierung der Waffe bestimmt Schussrichtung
 - o Treffen andere Mobs NICHT
 - o Haben Variablen, die ihr Verhalten bestimmen



Mob Spawning

Das Mob Spawning geschieht mithilfe eines Timers und eines Path2D's, welcher als Child vom Player existiert.

- Path bewegt sich mit Player

Betreten der Szene aktualisiert den Path2D einmalig

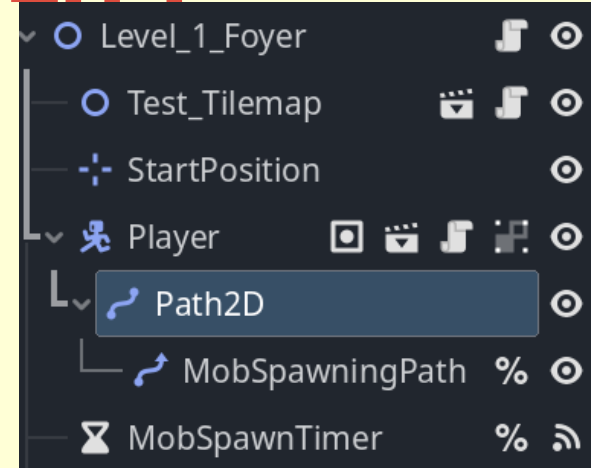
Nach jedem Timeout wird ein Mob gespawnt an einer zufälligen Position des Path's

- Position muss NavLayer haben bzw innerhalb der MobSpawnableTiles Layer existieren

- Timer wird bei jedem Timeout minimal kürzer

Spawnfunktion ist rekursiv aufgebaut

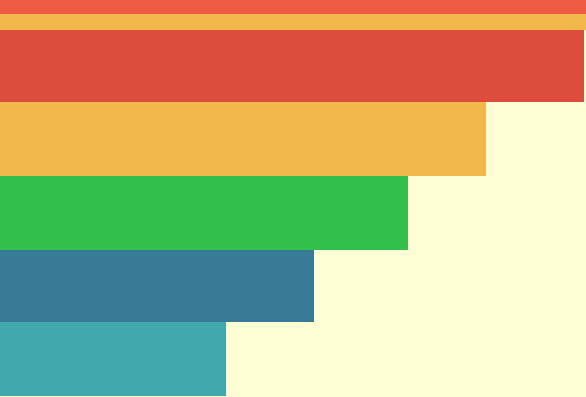
- Falls keine valide Position gefunden wurde
-> try again mit einer anderen Position
- Besitzt ein Limit für maximale Versuche



Mob Spawning Code

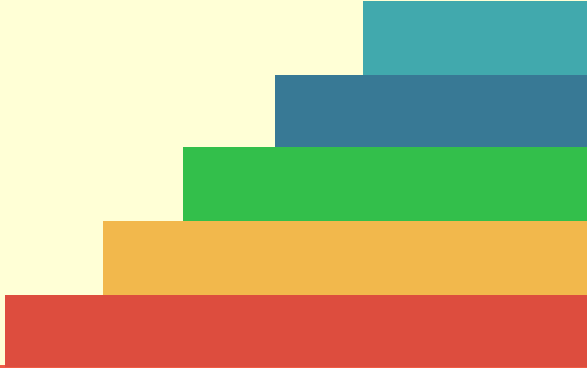
```
func spawn_mob(maxAttempts : int):
>| #check if there are less mobs than our mob limit currently spawned
>| if(current_mob_amount < MOB_LIMIT):
>| >| #generate a random number between 0-1
>| >| var which_mob = randf()
>| >| if (which_mob > 0.2): #80% chance for melee mob
>| >| >| var new_mob = load("res://Mobs/mob.tscn").instantiate()
>| >| >| #sets the var to a rdm position on its path
>| >| >| MobSpawningPath.progress_ratio = randf()
>| >| >|
>| >| >| #if that spawn position is within our navmesh -> spawn mob at that position
>| >| >| if(MapSpawningFloor.spawncheck(MobSpawningPath.global_position)):
>| >| >| >| new_mob.global_position = MobSpawningPath.global_position
>| >| >| >| add_child(new_mob)
>| >| >| >| current_mob_amount += 1
>| >| >| >|
>| >| >| elif(maxAttempts > 0):#do we have more attempts?
>| >| >| >| spawn_mob(maxAttempts - 1) #yes -> try again with different position
>| >| >| >| # -1 to avoid infinite loop
```

```
func spawncheck(coords: Vector2):
>| if (local_to_map(to_local(coords)) in get_used_cells()):
>| >| return true
>| return false
```



Ent_Boss

Gustav aka SirQuackAlot





Problemfrage:

Wie macht man einen komplizierteren Gegner
ohne vor lauter Funktionen die Übersicht zu
verlieren?



States

Ziel: Ein Gegner befindet sich immer nur in einem State. Jeder State hat seine eigene Funktion, wofür er existiert.

States sorgen für:

- Eine bessere Struktur
- Besser lesbaren/verständlichen Code
- Funktionen sind getrennt

Passend zu den States brauchen wir aber auch etwas, was sich um diese kümmert, und uns sagt in welchem State wir uns befinden

```
class_name State extends Node

signal Transitioned

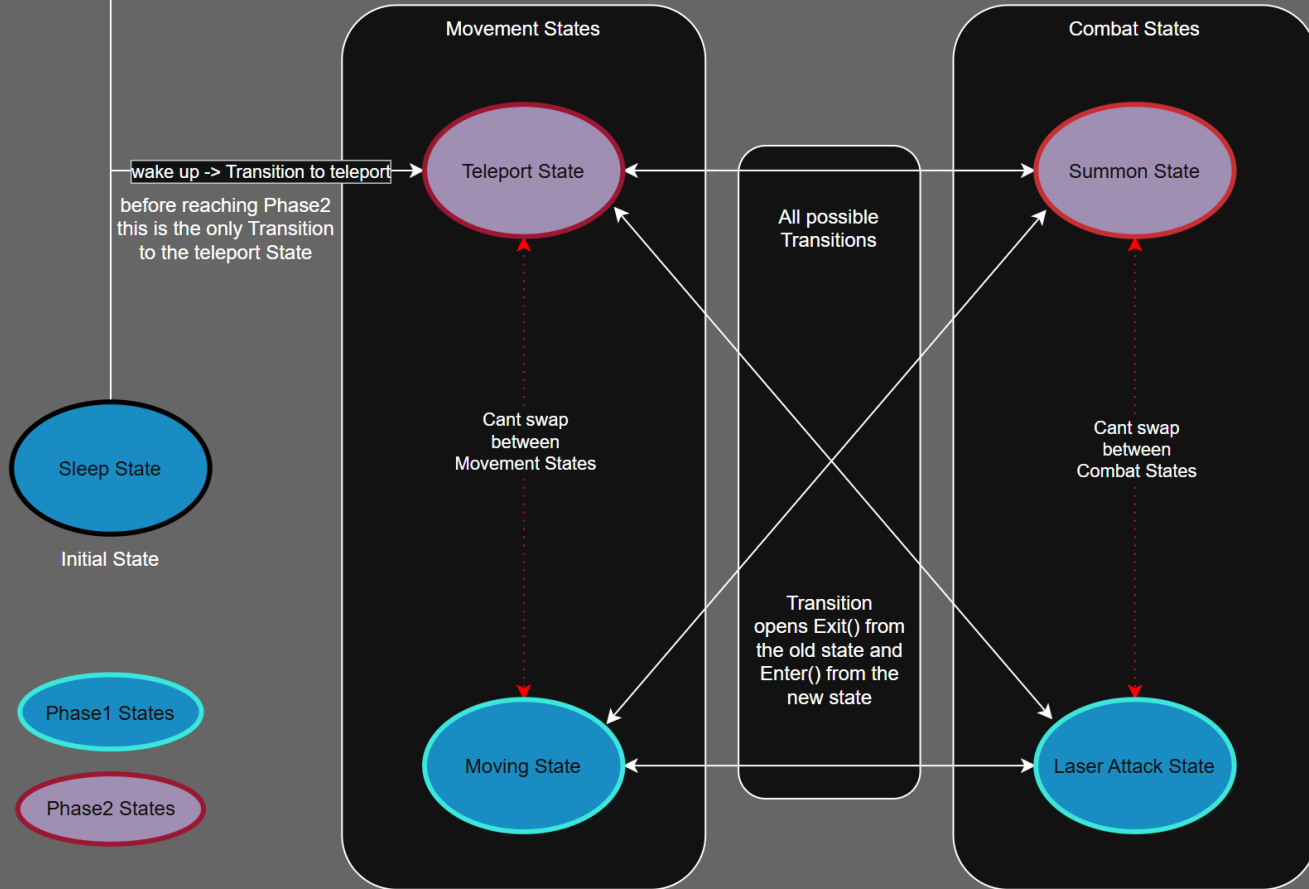
func Enter():
>| pass

func Exit():
>| pass

func Update(_delta: float):
>| pass

func Physics_Update(_delta: float):
>| pass
```


State Machine



State Machine Code

```
## Variables
@export var initial_state : State
var current_state : State
var states : Dictionary = {}

#Called when the node enters the scene tree for the first time.
func _ready():
    >| #setup dictionary of states
    >| for child in get_children():
    >| >| if child is State:
    >| >| >| states[child.name.to_lower()] = child
    >| >| >| child.Transitioned.connect(on_child_transition)
    >| >| >| #important signal of how we swap between states
    >|
    >| #set boss to initial state
    >| if initial_state:
    >| >| initial_state.Enter()
    >| >| current_state = initial_state

#calls the current state's update function each frame
func _process(delta: float):
    >| if current_state:
    >| >| current_state.Update(delta)

#calls the current state's physics update function each frame
func _physics_process(delta: float):
    >| if current_state:
    >| >| current_state.Physics_Update(delta)
```

Wie man zwischen States wechselt:

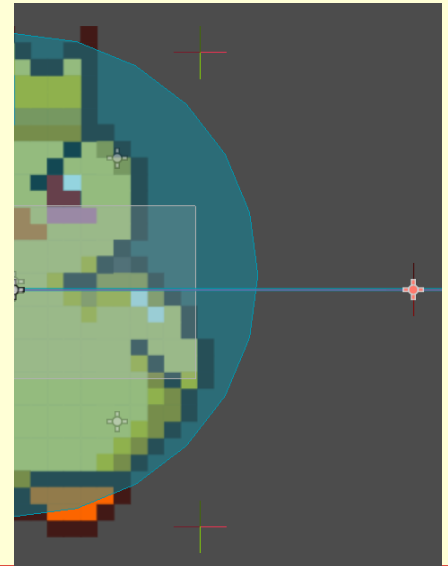
```
func on_child_transition(state, new_state_name):
    >| #if first param doesnt match current_state then stop
    >| if state != current_state:
    >| >| return
    >|
    >| #grab the state from our dictionary
    >| var new_state = states.get(new_state_name.to_lower()) #to
    >| if !new_state:
    >| >| return
    >|
    >| #call the exit function of our old state
    >| if current_state:
    >| >| current_state.Exit()
    >|
    >| #call the enter function of our new state
    >| new_state.Enter()
    >|
    >| #overwrite old state with new state in var current_state
    >| current_state = new_state
```

Extra Info

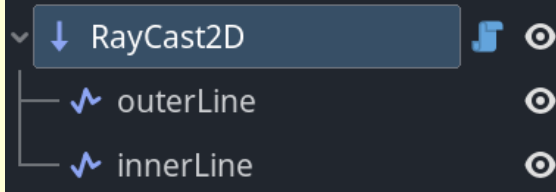


- Wandering & Follow States funktionieren werden jedoch nicht benutzt
- Boss bewegt sich NICHT mit move_and_slide()
 - o Verhindert dass der Spieler ihn schieben kann
- Moving State nutzt einen NavAgent2D
 - o Vordefinierte Navigationspunkte

- Summon State nutzt Marker2D's
 - o Mobs werden gezielt in Richtung des Spielers gespawnt
- Healthbar wird erst beim verlassen des Sleep States angezeigt
- Bei unter 50% HP wird Phase2 aktiviert
 - o Manche States kriegen stärkere Stats



BossLaser



Variables

```
@export var cast_speed := 7000.0#T
@export var max_length := 2000.0
@export var start_distance := 40.0
@export var growth_time := 0.1
@export var damage_rate := 5.0
```

Dies ist womit sich der Boss vor dem Spieler verteidigt

- Boss besitzt 3x Laser
 - o 2x für die Anfangs Spinattack + Standard Laserattack
 - o 1x sodass der Spieler ihm nicht zu nahe kommt

Laser ist ein Raycast2D – erkennt also Kollisionen

- 2x Line2D sind für Visuals
- Wird via Setter Funktion der Variable is_casting aktiviert oder deaktiviert
- Besitzt change_preset(String) Funktion
 - o Ermöglicht schnelle Stat Änderungen

Management & Codierung Selina

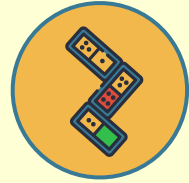


Projekt- ablauf

- Aufgaben Aufteilung
- Wöchentliche Ziele
- Merge Master

Code Management

- Scene Controller
- Globale Skripte
- Gruppen



Codierung

- Player
- Effekte
- Ducks
- Kleinere Aufgaben



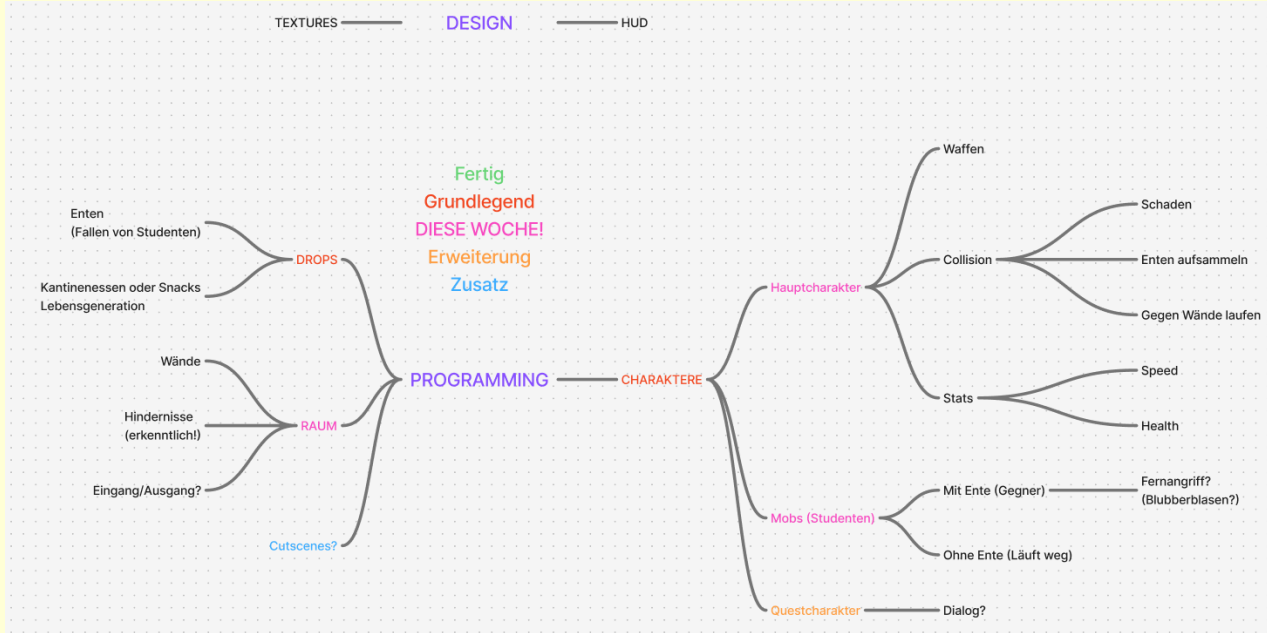
01

Projekttablauf

Wöchentliche Ziele & Updates

	Mon	Tue	Wed	Thu	Fri
Week 1	▪ Überlegen, welche Schritte als nächstes benötigt werden, um die großen Ziele zu erreichen ▪ Die Teams direkt ansprechen und Aufgaben anbieten oder verteilen ▪ Updates geben, was alles erreicht wurde ▪ Nach Feedback und Ideen fragen				
Week 2					
Week 3					
Week 4					

Nutzung von Figma



Merge Master

GitHub

- Sich einlesen und auf einen Vorgang einigen

Pull Requests

- Innerhalb 24h
- Code Konflikte lösen

Bugfixes

- Neue Versionen testen und Bugs fixen

Play!





02

Code Management

Scene Controller

Scene Controller



- Main Scene
- lädt und entfernt Nodes
- global zugreifbar

Game2D

- enthält die Level Nodes

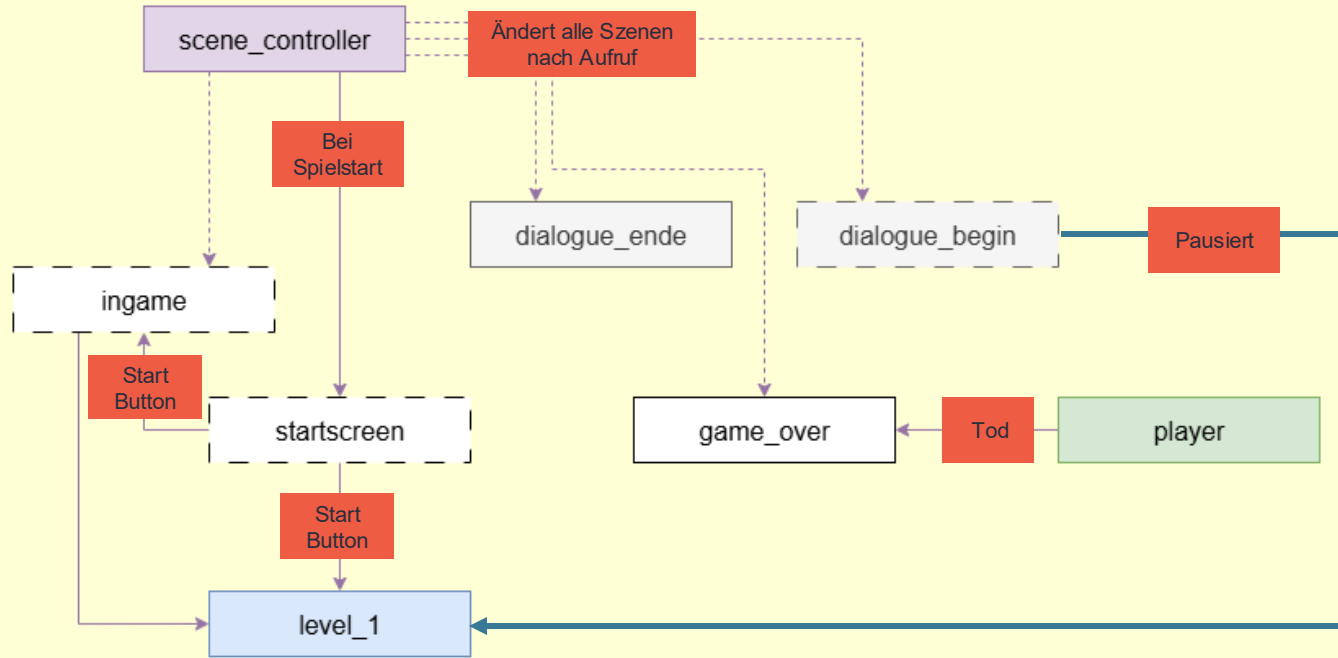
Control_GUI

- enthält Overlays und Huds

Sounds

- enthält globale Sounds

Scene Controller



Scene Controller

```
▼ func change_game_scene (new_scene_name: String, delete: bool = true, visibility: bool = false):  
    >| remove_scene(delete, visibility)  
    >| #loads new scene and starts it  
    >| var new_scene = load(new_scene_name).instantiate()  
    >| current_scene = new_scene  
    >| print("Current Scene: ", current_scene)  
    >| world2D.add_child(new_scene)
```

change_game_scene ▪ change_gui_scene ▪ remove_scene ▪ remove_gui

```
GlobalSignals.scene_controller.change_game_scene("res://CutScene/Dialogue_End.tscn")  
GlobalSignals.scene_controller.remove_gui()
```

Nach boss_slain Signal beim betreten des PC-Pools

Level Manager

Anzahl Gesammelte Enten

Alle 5



LevelUp

Player und Mobs
werden stärker!

MAX/2



Halfway

- Dialog Event

MAX



Gustav

- Dialog Event
- Aufzug aktiviert
- Bosskampf

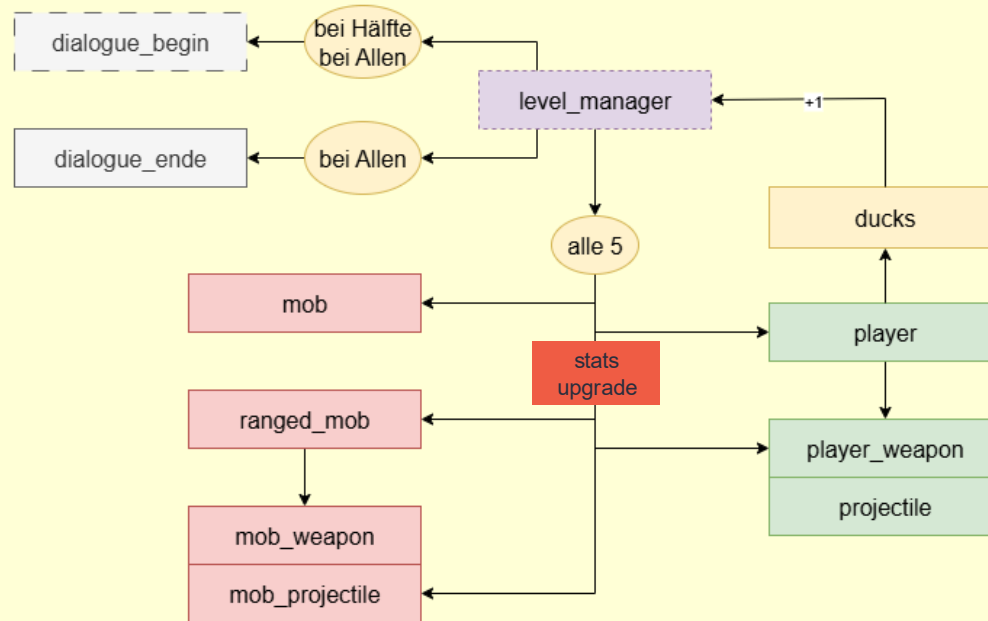
On Signal



game_over
boss_slain

Enten werden auf 0
gesetzt

Level Manager



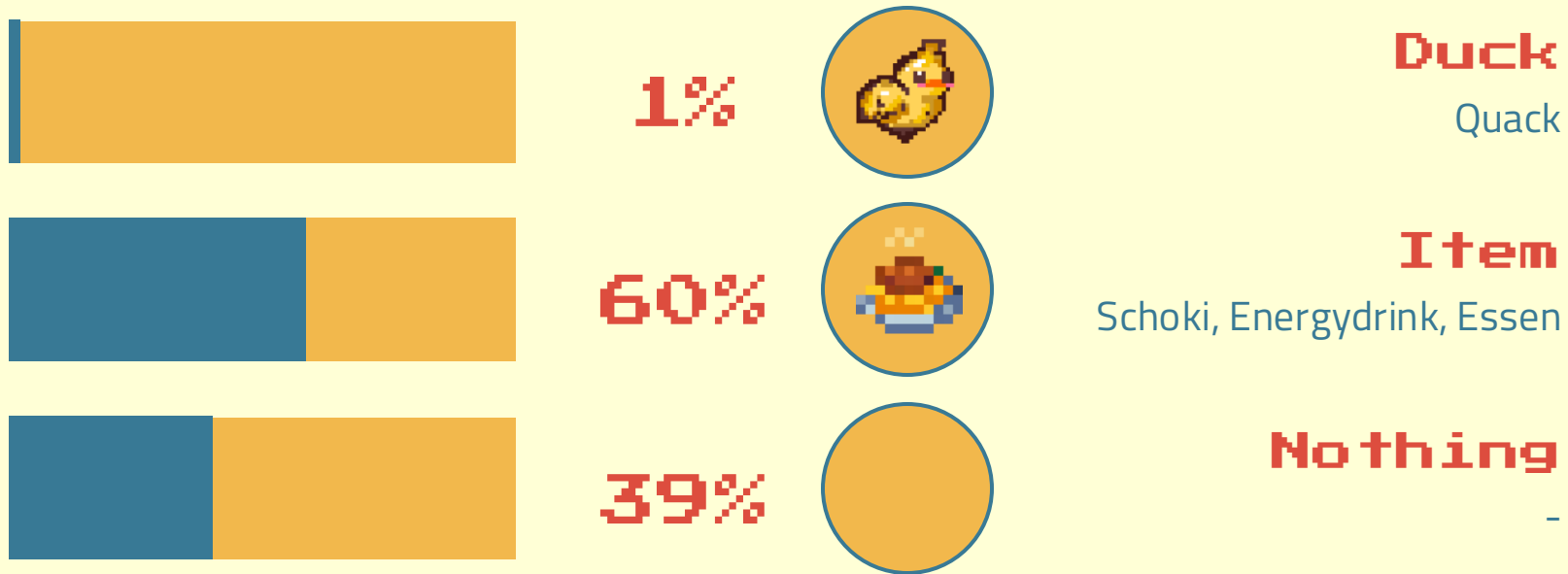
Level Manager

```
## IN LEVEL_MANAGER
func duck_collected_counter(amount: int = 1):
>|   ducks_collected += amount
>|
>|   if ducks_collected == ducks_required_gameWon:
>|       >|   GlobalSignals.dialogue_start.emit("res://CutSc
>|       >|   GlobalSignals.game_won.emit()
>|
>|   if ducks_collected == ducks_required_gameWon / 2:
>|       >|   GlobalSignals.dialogue_start.emit("res://CutSc
>|       >|
>|   if ducks_collected % ducks_required_levelUp == 0:
>|       >|   GlobalSignals.duck_collected_levelUp.emit()
```

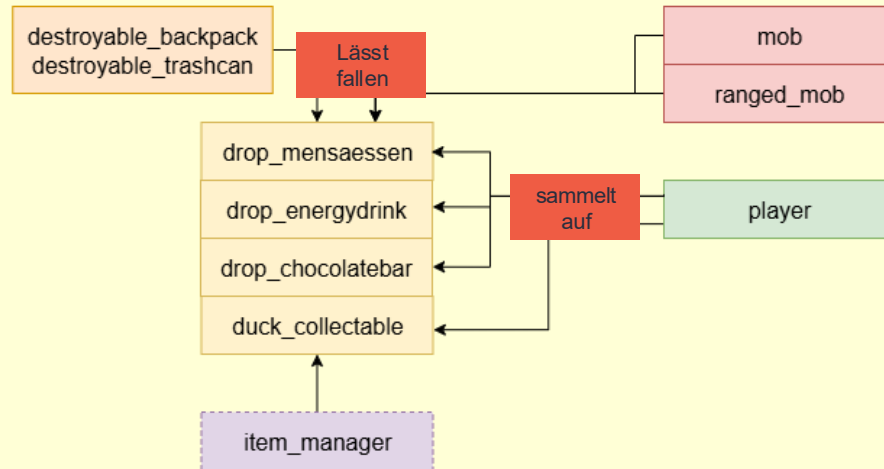
```
## IN DUCK_COLLECTABLE
func _on_collection_area_body_entered(body: Node2D) -> void:
>|   GlobalSignals.duck_collected_signal.emit()
>|   quack.set_deferred("pitch_scale", randf_range(1.1, 1.6))
>|   quack.play()
>|   animationPlayer.play("collected")
```


Item Manager - Beispiel

Chancen wenn man Studenten befreit



Item Manager



Item Manager - Beispiel

```
var ducks := preload("res://Drops/duck_collectable.tscn")
var chocobar := preload("res://Drops/drop_chocolatebar.tscn")
var energydrink := preload("res://Drops/drop_energydrink.tscn")
var mensafood := preload("res://Drops/drop_chocolatebar.tscn")

func _drop_item(spawn_global_position : Vector2):
>|   dropChance = randf()
>|
>|   if dropChance <= 0.6:
>|       >|   _drop(spawn_global_position, drops[randi() % drops.size()])
>|   elif dropChance <= 0.61:
>|       >|   _drop(spawn_global_position, ducks)
>|   else:
>|       >|   pass
```

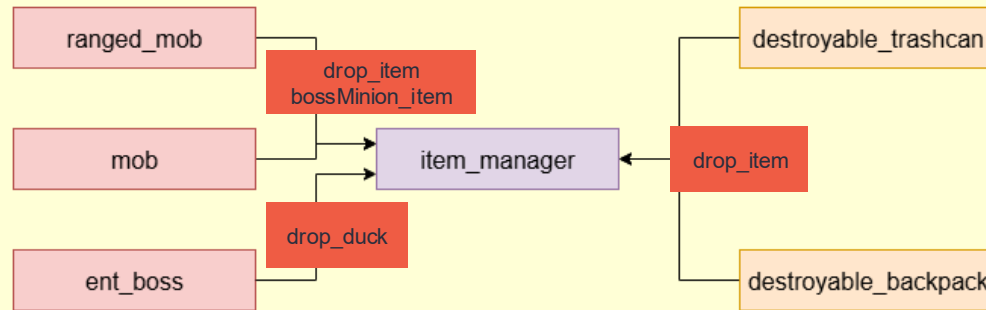
Mit Signal verbunden:
drop_duck, drop_item, bossMinion_item

Globale Signale - Doku

Skript	Sendet Signal - Dialoge (emit)	In der Funktion
dialogue_begin	dialogue_finished	_input() next_script()
player	dialogue_start (dialog_anfang.json)	[ONCE] _process()
	dialogue_start (ZwischenDialog_1.json)	[ONCE] _on_pick_up_area_entered()
level_manager	dialogue_start (ZwischenDialog_2.json)	[AT HALF] duck_collected_counter()
	dialogue_start (ZwischenDialog_3.json)	[AT ALL] duck_collected_counter()

Signal	Ist verbunden mit Funktion	Im Skript
dialogue_start	_pause_level_on_dialogue_start (dialogue_path)	level_1
	start (dialogue_path)	dialogue_begin
dialogue_finished	_unpause_level_on_dialogue_end()	level_1

Globale Signale - Beispiel





03

Code

Player

Player hat



- Level
- Leben
- Geschwindigkeit
- Waffe
- Collision Shape
- Pickup Shape (größer)
- Hurt Shape (ouch)



- Animationen
- Effekte

Player kann

- Boosts aufsammeln:
- Speed Up, Heilung, schneller Schuss

- Sich drehen
- Bewegen
- Schießen

- Sterben
- Gewinnen



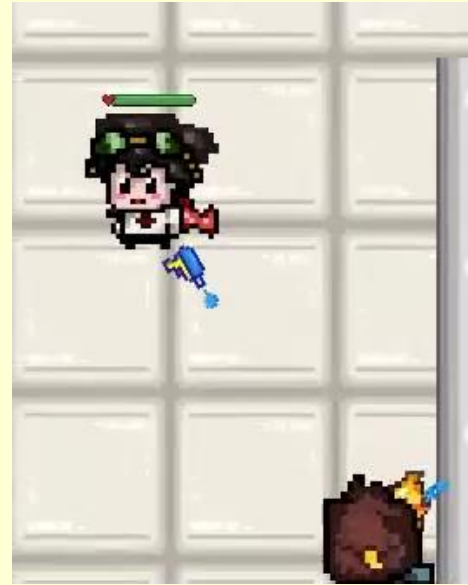
Collectable Ducks

Das Problem

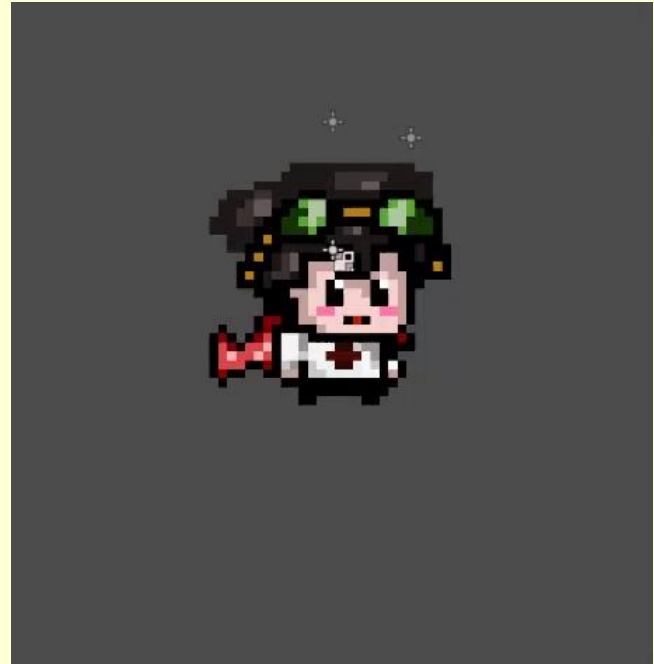
- Vom Gesicht der Studenten "runter" fliegen
- Kollision mit Objekten...

Die Lösung

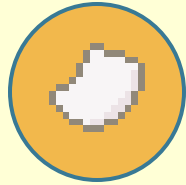
- Vordefinierter Flugpfad
- Bei Kollision: aus welcher Richtung...?
- In diese kann sich Ente nicht mehr bewegen!



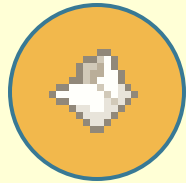
Effekte



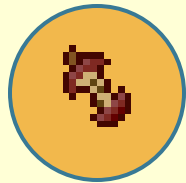
Effekte aller Art!



Gemacht via CpuParticles2D



Häufig via AnimationPlayer
mit anderen Effekten
verbunden



Nach abgeschlossener
Animation -> queue_free()





Vielen Dank für
Ihre
Aufmerksamkeit!

